MACHINE LEARNING TECHNIQUES FOR IDENTIFICATION OF COMMONALITIES AND SHARED ORIGIN OF LANGUAGE SCRIPTS

A Thesis

Presented to the

Faculty of

San Diego State University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

by

Vyshak Athreya Bellur Keshavamurthy

Summer 2019

SAN DIEGO STATE UNIVERSITY

The Undersigned Faculty Committee Approves the

Thesis of Vyshak Athreya Bellur Keshavamurthy:

Machine Learning Techniques for Identification of Commonalities and Shared

Origin of Language Scripts

Carl Eckberg, Chair
Department of Computer Science

Xiaobai Liu
Department of Computer Science

Samuel Kassegne
Department of Mechanical Engineering

05/15/2019
Approval Date

Copyright © 2019
by
Vyshak Athreya Bellur Keshavamurthy
All Rights Reserved

DEDICATION

I dedicate this thesis to my parents, Mr. Keshava Murthy B R and Mrs. Lakshmi P S. I thank them for their countless efforts and guidance and in helping me become successful. I would also like to dedicate this thesis to my friends, Abhishek Kadadi, Adil Rahman, Ramanuja Kalkunte, Kishore Bharatkumar, Ankit Mohanty, Vinod Mankar, Ashish Sahoo, Adithi Prasad and sincerely thank them for their endless support and optimism.

ABSTRACT OF THE THESIS

Machine Learning Techniques for Identification of Commonalities
and Shared Origin of Language Scripts
by
Vyshak Athreya Bellur Keshavamurthy
Master of Science in Computer Science
San Diego State University, 2019

Understanding unknown scripts, finding their origin and relating its relationship with the known scripts has intrigued linguistics. Many language scripts share common characters that are usually derived. Scripts imbibe from other scripts, differing only by small percentages. A scientific approach is being taken here to measure the structural similarity between different language scripts. Machine Learning techniques are used to compute similarity percentage. At first, most significant, set of features is extracted using the Principal Component Analysis, Independent Component Analysis, Factor Analysis and Non negative components. These features are compared using clustering algorithms such as K means and K nearest neighbour. Distance metrics such as Euclidean, Manhattan, Cosine, Chebyshev and Correlation are compared to choose the one that mimics the human visual system. Character wise similarity is measured with a mathematical score. Scripts are compared by number of characters that is similar between them with at least 50% similarity score. A pie chart is generated for each script indicating the percentage share. A tree structure is also derived from the distance metrics to estimate the character hierarchy in the same script and in between different scripts. Around 8 different scripts from different parts of the world are considered here. Through the writing system, human history has been recorded durably. Finding a similarity metric through Machine Learning techniques, will help discover and understand many language scripts worldwide.

TABLE OF CONTENTS

PAGE

ABSTRACT v
LIST OF TABLES viii
LIST OF FIGURES ix
ACKNOWLEDGEMENTS xi
CHAPTER
1 INTRODUCTION 1
Grapheme 2
Morpheme 2
Syllable 2
Functional Classification 2
Logograms 3
Syllabic System [Syllabary] 3
Segmental Systems [Alphabets] 3
Featural Systems 4
Ambiguous Systems 4
Directional Classification 4
Motivation 4
Organization of Thesis 4
2 LITERATURE SURVEY 5
3 DECOMPOSITION & CLUSTERING 8
Dataset 8
Decomposition 9
Principal Component Analysis 9
Non Negative Matrix Factorizations 10
Independent Component Analysis 10

Mini Batch Dictionary Learning 11
Factor Analysis 11
T-Distributed Stochastic Neighbor Embedding 12
Mini Batch K Means 14
Clustering 15
K Means 16
K Nearest Neighbor 16
4 FEATURE EXTRACTION & SIMILARITY LEARNING 19
Transfer Learning 19
Autoencoders 20
Local Binary Pattern 22
Uniform LBP 22
Similarity Learning 24
Histogram Intersection Distance 26
Log Likelihood Distance 26
Chi Square Distance 26
Correlation Distance 26
5 RESULTS & DISCUSSION 28
Further Enhancements 38
Summary 39
REFERENCES 40

LIST OF TABLES

PAGE

Table 1. Classification of Writing System 1
Table 2. Comparison of Decomposition Techniques 9
Table 3. Autoencoder Architectures 20
Table 4. LBP Computation 22
Table 5. LBP Types 22
Table 6. LBP Methods 22

LIST OF FIGURES

PAGE

Figure 1. Indus script stamps, seals and texts as referred in [4]. 5
Figure 2. Entropy of the Indus script compared to other natural languages. 6
Figure 3. PCA using randomized SVD. 10
Figure 3. First 6 components obtained using mini batch sparse PCA for Ethiopian script. 11
Figure 5. Non negative components for Ethiopian script obtained using NMF. 12
Figure 9. Pixelwise variance obtained through the factor analysis for the Ethiopic script characters. 12
Figure 6. Independent components generated for the Ethiopic script using fast ICA. 13
Figure 7. Components for the Ethiopian script obtained using mini batch dictionary learning. 13
Figure 8. Factor analysis components obtained for Ethiopic characters. 14
Figure 10. Cluster centers for the minibatch k means learning. 15
Figure 11. K means, and mini batch k means difference. Source: [20]. 15
Figure 12. Latin script classification with K=6 for first 40 components. 16
Figure 13. Latin script classification with K=6 and first principal component. 17
Figure 14. Clustering of Devanagari characters with K=4 and first principal components. 17
Figure 15. Classification of Devanagari and Ethiopic characters with K = 6 and first 5 principal components. 18
Figure 16. Results using Vgg19 Transfer learning for a Bengali query image. 19
Figure 17. Image similarity for Ethiopic-Devanagari characters using transfer learning. 20
Figure 18. Similar images fetched using autoencoders. Distance metrics made a difference. 21
Figure 19. Similarity using Autoencoders. Distance metrics had no effect. 21
Figure 20. LBP variations. 22

Figure 21. ror LBP values for Devanagari character. 23
Figure 22. var LBP values for Devanagari character. 23
Figure 23. Default LBP values for Devanagari character. 24
Figure 24. Uniform LBP values for Devanagari character. 24
Figure 25. Scatter plot for Ethiopian script for Intersection distance. 25
Figure 26. Scatter plot for Ethiopian script for Chi-square distance. 26
Figure 27. Scatter plot for Ethiopian script for correlation distance. 27
Figure 28. Scatter plot for Ethiopian script for Hellinger distance. 27
Figure 29. Image unfoldment as seen for the data set of Devanagari and Ethiopic characters. 28
Figure 30. An attempt to compare mean and variance of different language scripts. The first character in the top left of the query image is different. Yet the principal components are the same. 29
Figure 31. (a) - (d) Parts of the hierarchy tree zoomed in. (e) The entire character hierarchy tree for the Devanagari script. 30
Figure 32. (a)-(c) zoomed in images of the Ethiopic tree architecture. (d) Whole tree structure for the Ethiopic characters. 34
Figure 33. t-Sne plot of all the images in the dataset. 37
Figure 34. LBP computation and similarity measurement for Devanagari Ethiopian characters. 37
Figure 35. Character to character match measurement using the cut off of 50% for the Kannada language script. 38

ACKNOWLEDGEMENTS

I would like to sincerely thank my thesis chair, Dr. Carl Eckberg for providing me an opportunity to work on this thesis. His advice and valuable inputs have helped me attain the results. A big thanks to my thesis advisor, Dr. Sam Kassegne, of mechanical department for his guidance, encouragement and support throughout my research. His wealth of knowledge and confidence in me is what has kept my research going till the end. His diligence in providing all the possible resources and funds to complete this thesis is greatly appreciated. Also, I would like to thank and Dr. Xaobai Liu for providing me his valuable inputs.

CHAPTER 1

INTRODUCTION

Writing systems have been a means of communication in visual mode ever since prehistoric times. These have served as evidence of trade, art, and culture of those times. Writing systems have evolved through many folds since the Bronze Age of the 4th millennium BC. Before this, the pictograms, ideograms and mnemonic symbols, also known as proto-writing, constituted the means of communication. Different writing systems have been developed along partially independent lines differing in terminology. According to [1] a writing system:

1. Is usually associated with at least one language.
2. Can be combined in different proportions across multiple languages. (E.g. Arabic) or a language can have several writing systems (E.g. Hindi, Urdu)
3. Is developed over a sporadic timeline. Once established, they seldom change. Hence, preserve language features, expressions, and structural inheritances.
4. Comprises base elements called symbols. It is a collective permutation of these symbols that form a script.
5. Can be complete or partial based on how much they can express the spoken language

Writing systems can be broadly classified as Alphabets, Syllabary, and Logography.

Table 1. Classification of Writing System

Attributes	Comprises of	No. of symbols
Alphabets	Symbol/grapheme of consonant/vowel	20 - 35
Syllabaries	symbol of syllable/mora	80 - 100
Logograms	Each character representing a word/morpheme/semantic unit	>= 100

The constituents of writing systems which helps in its classification are of key importance. As a phoneme marks the basic unit of the speech system, so does the grapheme in the writing system.

GRAPHEME

Grapheme is the basic unit of any writing system (similar to phoneme in speech). It can be defined as minimally significant element, which taken together, form the building block to build writing systems. The difference various combinations of graphemes is also due to the rules of correspondence, also referred to as the grammar. E.g.: English's grapheme consists of 26 majuscule and miniscule symbols, punctuation marks and a few other symbols for numerals. An individual grapheme may be represented in various ways which are visually distinct. These are however interpreted as the same grapheme. Variations are known as allographs (similar to allophone in speech)

The terms glyph, sign and character are also used to refer to graphemes in writing systems such as cuneiform, Mayan, and Chinese. Graphemes can be

- Linear: made up of lines and strokes, or
- Non-linear: these consists of other marks as in Braille.

MORPHEME

It is a meaningful morphological unit of a writing (language) system, that cannot be further divided. It is a smallest grammatical unit in a language. Morpheme differs from a word, in the sense that word makes sense independently. A Morpheme might/might not make complete sense without additional suffix/prefix

SYLLABLE

These can be seen as a bridge which connects a writing system to the phonetics. They indicate the pronunciation patterns. The weight, stress points, intonations and other aspects which help in the vocal rendering is accounted for through syllables. They play a major role in characterizing different forms of poetry giving rise to meter. A syllable is also referred to as a mora. In a deeper linguistic sense, a mora differs in that it can consist, of two or more syllables.

FUNCTIONAL CLASSIFICATION

Writing system components, with a deeper understanding can be more than alphabets, syllabaries and logograms. Though there are many versions based on different linguistic research, the following are of major significance.

Logograms

These are the types where a single symbol represents an entire word. So, there are as many symbols as the number of words, in a language. Since the meaning is inherent to the symbol, the same symbol can be used in different languages. E.g. the Japanese use Chinese logograms. Numbers are best examples of a logogram. Symbol 1 is used in many languages. Whether it is called one or Uno depends on the language. Types of logograms are:

- Ideograms: symbols that represent abstract ideas. These may be words/ morphemes. Not widely used.
- Lexigram: symbol that represents a word which may/may not be indicative of a particular object.

Most modern logographic writing system is the Chinese. Its influence can be found in Sinitic (Sino-Tibetan), Japanese, Korean, Vietnamese and other east Asian languages.

SYLLABIC SYSTEM [SYLLABARY]

A set of written symbols that represent syllables. In a syllabary, there is no symbol to represent the common character in different sounds. E.g. among symbols to represent \ke\,\ka\ there is nothing common to represent k. The greater the number of vowels and consonant variations in a language, the greater will be the syllabary. E.g. Cree syllabics (versions of Canadian aboriginal syllabics), Mycenaean Greek, Cherokee scripts. Cree and Cherokee are native-American languages.

SEGMENTAL SYSTEMS [ALPHABETS]

An alphabet is a small symbol set, that historically represents phoneme of a spoken language. The first alphabet that was developed is called Abjad [2]. Abjad had symbols only for consonants. A few abjads like Arabic and Hebrew make use of vowels. E.g. Malay, Indonesian, Arabic. Abugida [2] is an alphabet in which symbols indicate the consonants with an inherent vowel. E.g. a symbol which represents ‘k’ can be modified to represent ‘ka’ and ‘ke’. Most Abugida are derived from abjads. Vowel modifications may not be systematic like in Geez script. E.g. Brahmic family of scripts [3]. Syllabary is a set of symbols that represent syllables. This differs from abugida, in the sense that syllabary has unique distinct symbols per syllable. E.g. Canadian Aboriginal Syllabics.

FEATURAL SYSTEMS

In featural systems, the symbols do not represent phoneme as a whole. They indicate the elements that make up these phonemes. E.g. Korean hangul. It features advanced graphic design corresponding to phonologic properties. Even the Latin script has sub-character features [4].

AMBIGUOUS SYSTEMS

Most writing systems are of composite types. Scripts such as Spain, have roots from syllabic scripts for plosives and alphabets for the rest. There are different opinions on how they should be classified.

DIRECTIONAL CLASSIFICATION

Scripts are also classified based on the direction in which they are written. It can be either left-to-right (LTR or sinistrodextrally) or right-to-left (RTL or dextrosinistrally). Before standardization, it was common to write in both directions. Usually, starting in one direction and reversing the direction at the end of the line.

MOTIVATION

There are a lot of similarities that are found in the middle eastern languages. These languages are not digitized. Few are still in the process of discovery. This applies to not just the middle east, but different parts of the world as well. When a new language is discovered, it is much easier to relate it to the known languages. This attempt is hoping to understand the similarities that exists between the different languages.

ORGANIZATION OF THESIS

The document is organized as follows: Chapter 1 provides the introduction to the topic and lays foundation for the work. Chapter 2 describes similar work that is being done both in the field of literature and machine learning. Chapter 3 analyses the approaches taken, in terms of the decomposition and clustering. Chapter 4 is about Feature extraction and similarity learning. Chapter 5 provides an elaborate discussion on the results and possible enhancements to the research.

CHAPTER 2

LITERATURE SURVEY

Lot of research has been put into character, author and word recognition. Owing to the popularity that the Optical Character Recognition has, it was implemented to identify language characters accounting for environmental variations such as text images, hand written characters, noisy images etc.

Researchers have found that scripts such as the Southern Brahmi, Kannada, Pallava and Vatteluttu are related to each other. scripts, which in turn diversified into other scripts of South India and Southeast Asia [5]. There are evidences that prove that Telugu script derived from Bhattiprolu Script or "Kannada-Telugu script [3, 6].

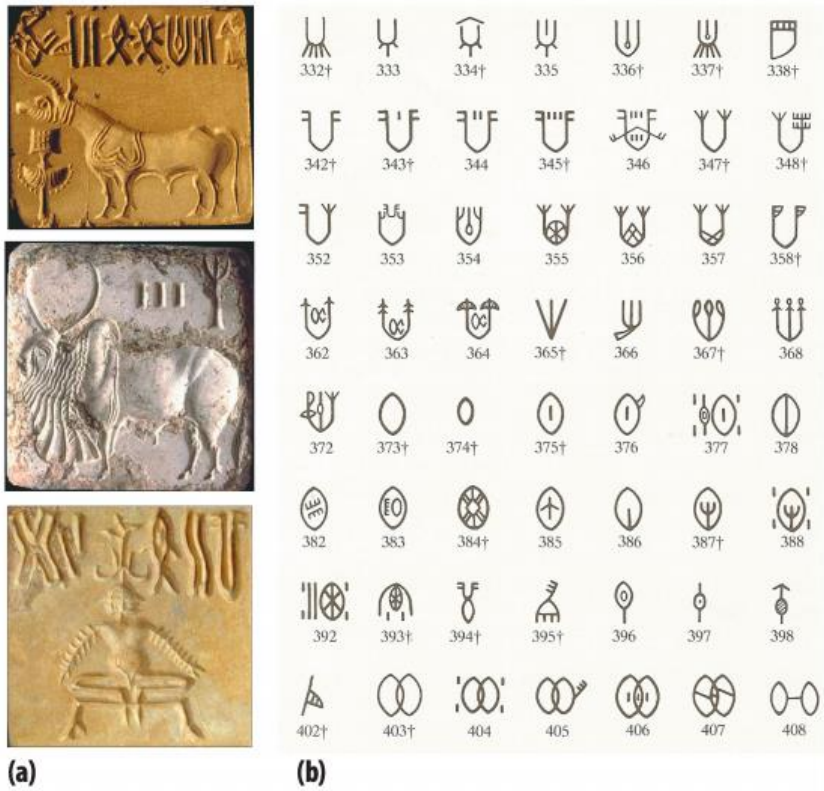


Figure 1. Indus script stamps, seals and texts as referred in [4].

Log likelihood, is used by them as a probability measure of the next occurring character, given the first character. Other parameters such as the average length and mean are also compared. With this as prime measure, they compute the entropy between the languages. The study gives an estimate of the influence and similarity of different languages with the Indus script.

Rao et al did a probabilistic analysis of ancient undeciphered scripts [4]. They found exemplary results when doing a archaeological study of the Indus script. In their study they have used Machine Learning to compare entropy between different ancient scripts and categorizing those based on the entropy levels. With the help of the Markoff models, they generated closely matching text to the Indus script and measured the log likelihood. They further continued the research by measuring the log likelihood with different foreign scripts.

There are several other researches that have been done to analyse the language scripts and generate Unicode. Using machine learning techniques, many experiments have been done using Optical Character Recognition (OCR) on different character recognition [7] and author identification [8].

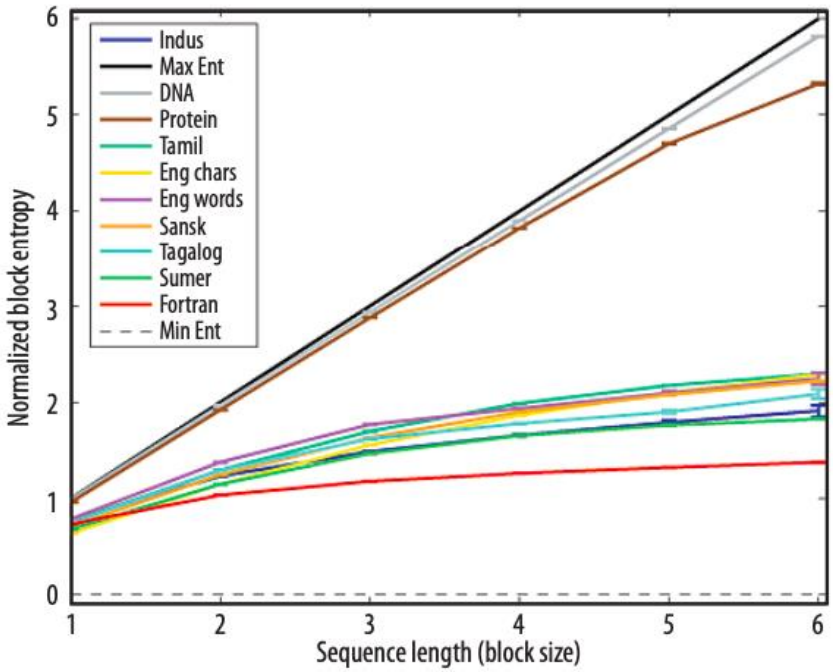


Figure 2. Entropy of the Indus script compared to other natural languages. Source: [4].

Implementation of the ML techniques for the classification is humungous. wolf et al - insect classification. Wan et al performs gesture recognition using SIFT to obtain hierarchical feature extraction. The computed features fed to nearest neighbor classifier. Lake et al comes up with a generative hierarchical hidden Markov model inclusive of a Bayesian inference to classify speech primitives. Maas and Kemp uses Bayesian network to generate meaningful information from the Ellis island passenger data. Wu and Dennis derive a path planning algorithms for robotic actuation so that the energy function that can be minimized. Lin et al applies the transferring learning across object categories to borrow examples from other classes in the training set [9].

Finding structure in randomness [10] paved way for the present-day similarity recognition measures. The authors discuss a crude but effective way to identify similar images. A big application of this has been in the face recognition. Using Principal Component Analysis, eigen faces are generated. These represent the core features of a face. All the other images are variations of the core eigen face. So, the amount of variation differentiates the faces. [11] builds on the concept to develop a face recognition system.

The technique of deep image retrieval is useful in getting similar images to the search image. An implementation of this is done in where they build the region wise descriptors. They have a ranking framework which uses weights and convolution [12].

Other approaches with deep convolution networks to recognize images have been done in [13]. Using this network as a feature extractor, many similarity measure systems have been developed. Once the features are extracted, the distance between the query image and the search image is measured [14] and similar images are retrieved.

In a broader perspective, all the approaches that are taken follow the path of feature extraction and similarity measurement. One of my favorite techniques of feature extraction was described in [15] as local binary patterns. These are implemented in [16] to recognize faces. Authors claim that their system out forms the complicated neural networks by a large extent.

CHAPTER 3

DECOMPOSITION & CLUSTERING

Human visual system has very high sensitivity to finding similarity between objects. Based on the similarity, many new things are learnt as well. The process of decomposition is which helps machine learning models to identify the data distribution space. If machine can get the mean and how much variance other data is having, it would help in obtaining the similarity.

DATASET

There are many datasets that are created for different language scripts. [17] is a collection of all the Indus script symbols and seals. Similarly, Omniglot [18] has a database of 50 languages with 20 characters from each language. The problem with selecting those databases was unfamiliarity of those languages. Another reason was that there were only 20 characters and all were not selected based on the similarity.

An attempt was made to generate the characters using PIL python library from the Unicode. Reasoning behind was that we can have custom positions for the start and end of the characters which would be a good starting point to measure the similarity. It was successful for the Latin characters for which the library could accommodate. The same script was used to generate using ttf files. Limitation was again for the non-Latin characters.

Next step in our dataset search was to take the screen shots. The images that were generated from the ttf files were not of a good resolution. Though there are many charts and images for languages scripts, it’s not exhaustive. Hence using online language editors such as Lexi logos [19] are used to generate high resolution images. These were then reframed to 64*64, 240*320 and 256*256 sized images.

DECOMPOSITION

The process of getting the distribution of the data across different dimensions is known as decomposition. There are many different techniques that can aid in the process of decomposing images. We implemented those using python scikit-learn library.

Principal Component Analysis

In the process of analysis, the given character or the image is broken into vectors. These are high dimensional vectors. To find a pattern in a high dimensional data is difficult and hence the dimension is reduced ensuring that the pattern is preserved.

The input data may not be the same image or character using which the network was trained. It may differ in a way that it may be skewed, scaled or in a different shape al together. PCA helps in these cases as the reduction in dimension maps down to almost similar patterns that other forms of the same image would produce. The data whose dimension is thus reduced, is given as an input to the neural network.

In the scikit learn library, there are different methods of PCA implementation: Singular Vector Decomposition, mini batch sparse PCA, kernel PCA and incremental PCA

We experimented using the SVD and mini batch sparse PCA methods on the Ethiopic dataset. The results are as shown in figure 3 and 4. First 6 components are being displayed. In terms of the efficiency of the methods, we also computed the time taken. It is summarized in Table 2.

Table 2. Comparison of Decomposition Techniques

Decomposition technique	Training time
Principal Component Analysis using Randomized SVD	0.4s
Mini batch Sparse PCA	95s
Non negative components	0.9s
Mini batch dictionary learning	6.9s
Fast Independent Component Analysis (ICA)	0.7s
Factor Analysis	1.6s
Mini batch K means	0.9s

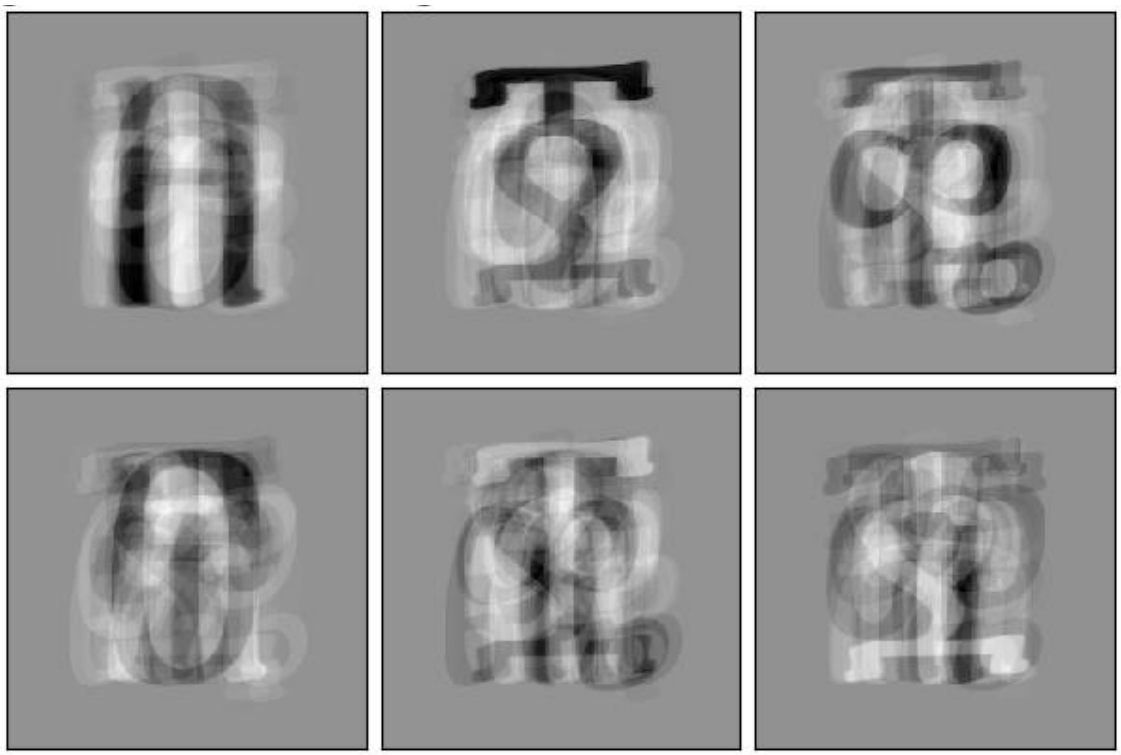


Figure 3. PCA using randomized SVD.

Non Negative Matrix Factorizations

It is a matrix decomposition technique where in a matrix M is decomposed to 2 non negative matrices M and N such that the difference between X and the product MN is minimal. To measure the distance, squared Frebenius norm is used. When we used this for the Ethiopian dataset, the results are as shown in Figure 5.

Independent Component Analysis

This is an approach that is used to decompose the signals that are superimposed on one other. More than a decomposition technique, this can be seen as a process of extracting the smaller elements that make a bigger picture but without any means of dissection. The results from this technique is shown in Figure 6.

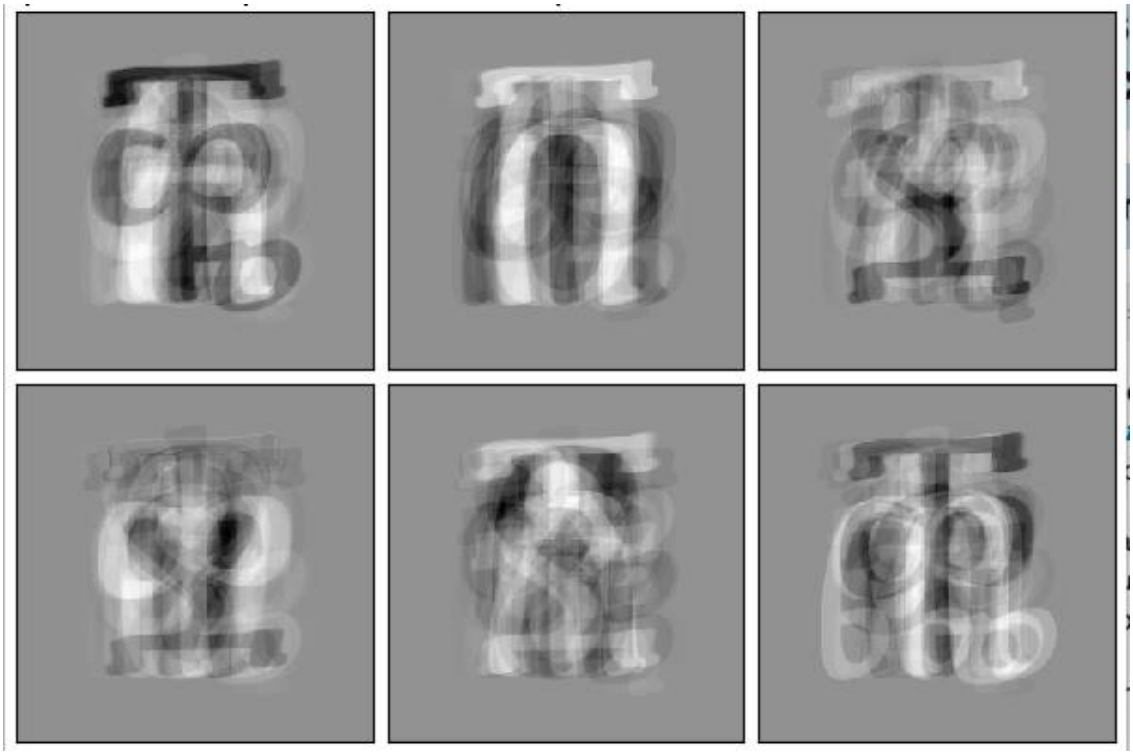


Figure 3. First 6 components obtained using mini batch sparse PCA for Ethiopian script.

Mini Batch Dictionary Learning

A process which can act like a test case for the reduced features is this mini batch dictionary learning. Dictionary is a feature such as wavelet basis. The evaluation process is carried out in ensuring that the low dimensional features represent the data as good as all the features. Results are in Figure 7.

Factor Analysis

This technique is similar to Principal Component Analysis. They differ by the fact that Factor Analysis (FA) will not always produce reduction in orthogonal dimensions. A plus factor for the FA is that it provides the pixel wise variance in all the dimensions as shown in Figure 9. Results of the FA as applied to the Ethiopic script is shown in Figure 8.

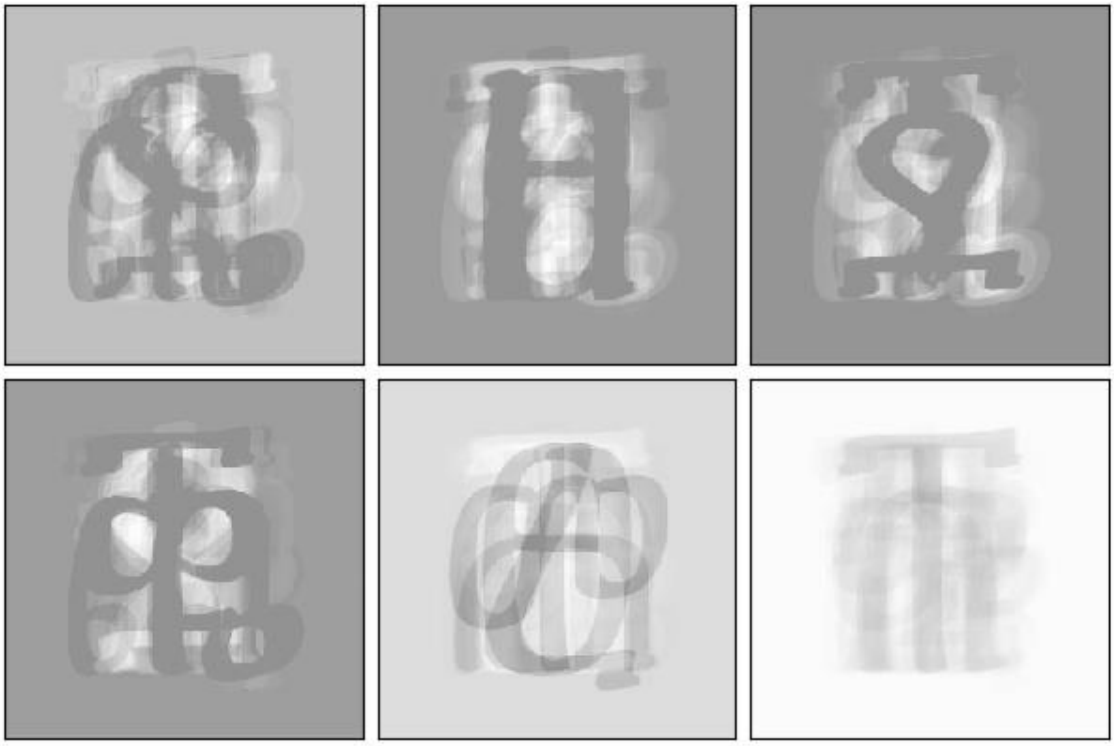


Figure 5. Non negative components for Ethiopian script obtained using NMF.

T-Distributed Stochastic Neighbor Embedding

Another useful technique to reduce the high dimension data into a low dimensional space. This is very useful to get a visualization of the data distribution. In terms of efficiency, it is not on par with the better techniques such as the PCA.

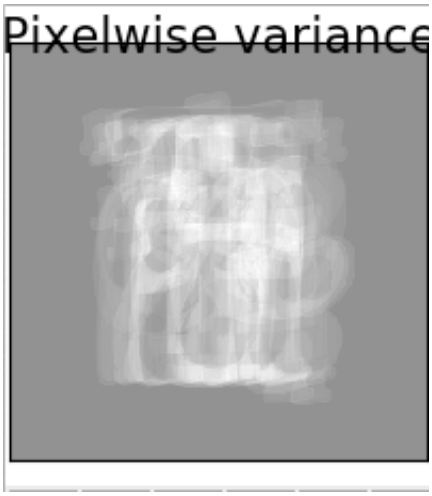


Figure 9. Pixelwise variance obtained through the factor analysis for the Ethiopic script characters.

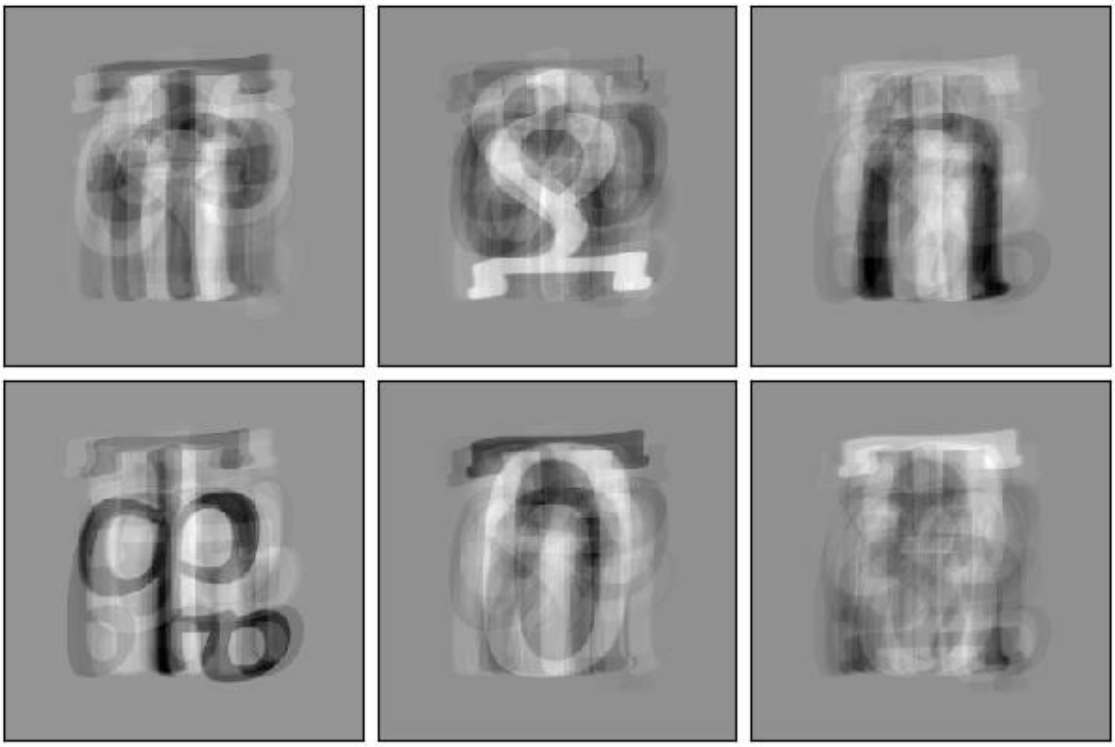


Figure 6. Independent components generated for the Ethiopic script using fast ICA.

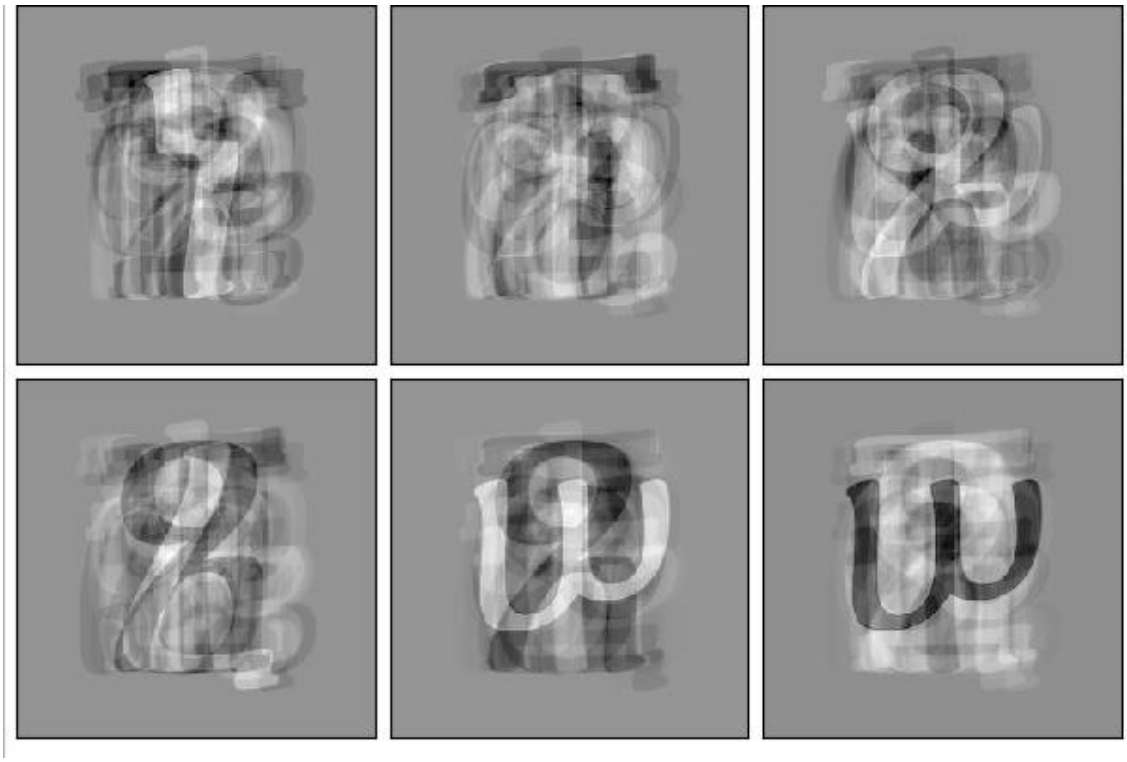


Figure 7. Components for the Ethiopian script obtained using mini batch dictionary learning.

T-Distributed Stochastic Neighbor Embedding (TSNE) can act as a great tool to visualize the data distribution across the given dimensions. This helps us to find the clusters to which similar data belongs.

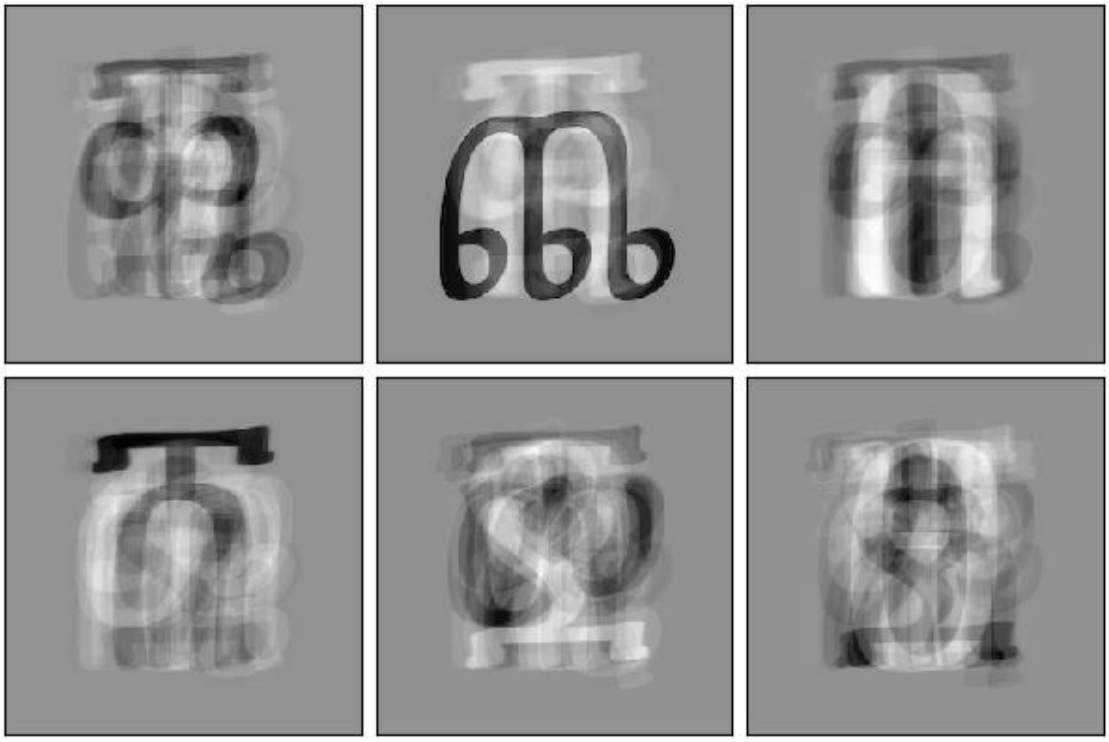


Figure 8. Factor analysis components obtained for Ethiopic characters.

Mini Batch K Means

Although K means is a clustering algorithm, mini batch k means is used as a dimensionality reduction technique. Major difference between the vanilla k means and the mini batch is in the way they are computed. Mini batch follows the same techniques in computing the centroids, assigning the groups and converging. The results for the Ethiopian character set is as shown in the Figure 10.

The results from the k means and mini batch k means can be fed into the distribution visualization tools such as TSNE or the h cluster tree algorithms. These take the groups and develop a tree hierarchy. Tree hierarchy is usually based on the distance parameter. Experiments have been performed to generated the language hierarchy tree.

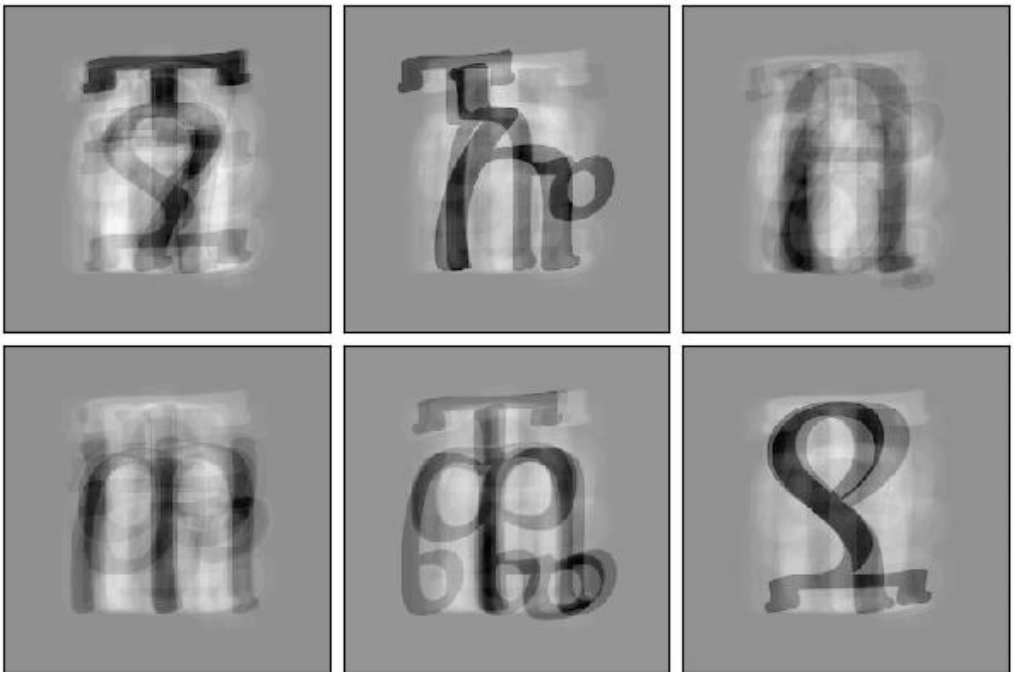


Figure 10. Cluster centers for the minibatch k means learning.

CLUSTERING

The process of finding an order in the data set and forming relative, relevant groups within them is known as clustering. As like the decomposition, there are many techniques that are available for the clustering. In this project, we have used K means and KNN clustering algorithms.

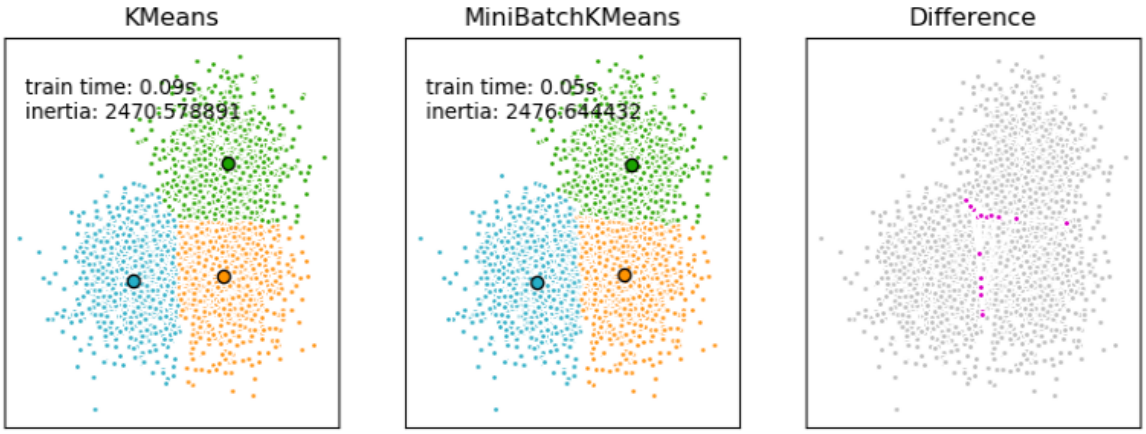


Figure 11. K means, and mini batch k means difference. Source: [20].

K Means

An iterative algorithm, that tries to group different data points based on certain distance metric. Initially random points are chosen as centroids. Number of centroids is the value of K. The distance between the data points is computed and groups are formed. For the next iteration, new centroids are computed and the process repeats.

K means doesn't work well in a high dimensional data environment. To overcome this, it is experimented on the data that is decomposed by techniques such as PCA. We performed K means experiments on combinations of different data scripts with only the characters in the script. For different values of K and different iterations, we got interesting results. These are shown from Figure 12-16.

K Nearest Neighbor

This algorithm is a weighted nearest neighbor classifier which works on both regression and classification problems. Data points are classified based on the votes assigned to them. KNN is implemented on the output of autoencoders. It is discussed in the next chapter.

Figure 12. Latin script classification with K=6 for first 40 components.

Figure 13. Latin script classification with K=6 and first principal component.

It can be observed that the characters have appeared in more than one group. There is no duplication in the dataset. As an enhancement, the PCA features were compared to the clustered data and it was found that those were similar to the corresponding principal components.

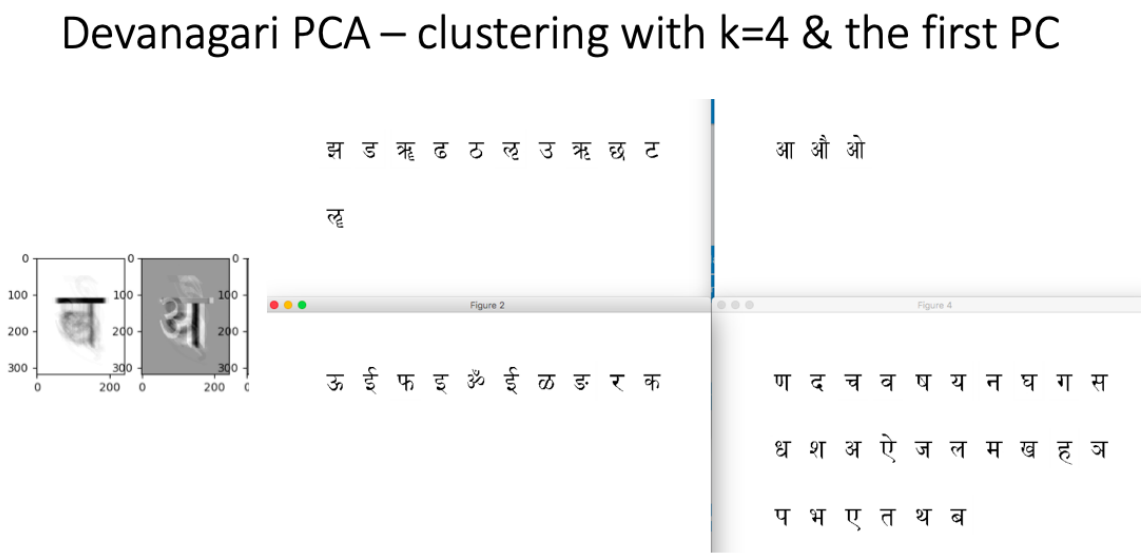


Figure 14. Clustering of Devanagari characters with K=4 and first principal components.

Clustering experiment with different values of k and principal component, differed from script to script. Choosing one value of k which works for all is challenging. However, if fewer principal components are chosen, the results are found to be better.

Figure 15. Classification of Devanagari and Ethiopic characters with K = 6 and first 5 principal components.

CHAPTER 4

FEATURE EXTRACTION & SIMILARITY LEARNING

Good features are the primary key in the field of machine learning. Obtaining those efficiently will increase the overall productivity of the system. In this thesis, there are 3 feature extraction algorithms that have been used: Transfer Learning, Autoencoders and Local binary patterns.

TRANSFER LEARNING

The process of extracting features from existing, pre-trained neural networks for the purpose other than it was initially trained for is known as transfer learning. Vgg networks are popular neural nets what won the ImageNet competition. [13] implements different architectures of the Vgg with different layers. For our data set, we experimented with Vgg 16 and Vgg 19. Since the pre trained weights didn't give good results, we trained the network from the scratch. Testing efficiency was 91%. Sample of results obtained from transfer learning are shown in Figure 16 and 17.

Figure 16. Results using Vgg19 Transfer learning for a Bengali query image.

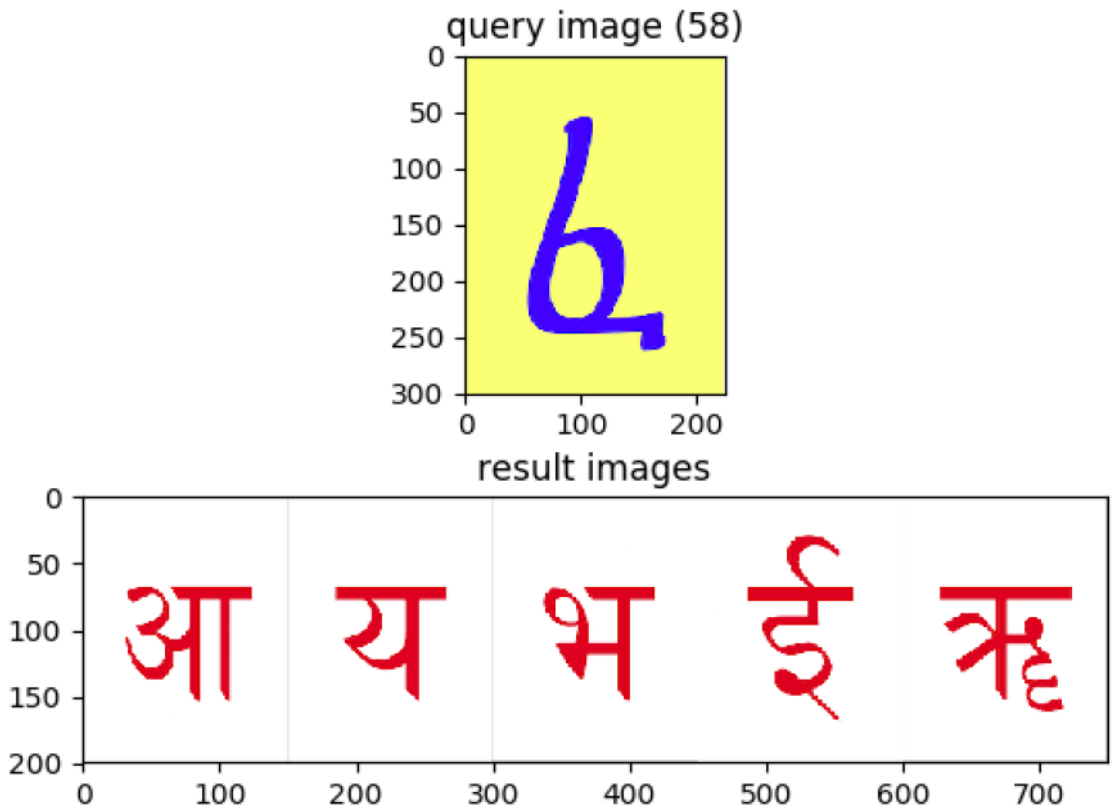


Figure 17. Image similarity for Ethiopic-Devanagari characters using transfer learning.

AUTOENCODERS

These are a collection of 2 neural networks connected face to face. Input is fed to the first part of the network called as encoder. Output of encoder is the compressed data. This is again fed to the second network i.e decoder. The compressed set of features is then fed to a KNN model. Based on the prediction from the KNN model similar images are retrieved. Different auto encoder architectures were experimented. Their summary is in Table 3.

Table 3. Autoencoder Architectures

Architecture	train error	test error
n_hidden_1 = 16 # 1st hidden layer n_hidden_2 = 16 # 2nd hidden layer n_hidden_3 = 8 # 3rd hidden layer	5.59%	5.74%
n_hidden_1 = 32 # 1st hidden layer n_hidden_2 = 32 # 2nd hidden layer n_hidden_3 = 16 # 3rd hidden layer	5.63%	5.91%
n_hidden_1 = 32 # 1st hidden layer n_hidden_2 = 16 # 2nd hidden layer n_hidden_3 = 16 # 3rd hidden layer n_hidden_4 = 8 # 4th hidden layer	4.93%	5.49%

Autoencoder features were also compared with different distance metrics which are explained in detail in the next section. For auto encoders, pairwise distance and cosine distance were used.

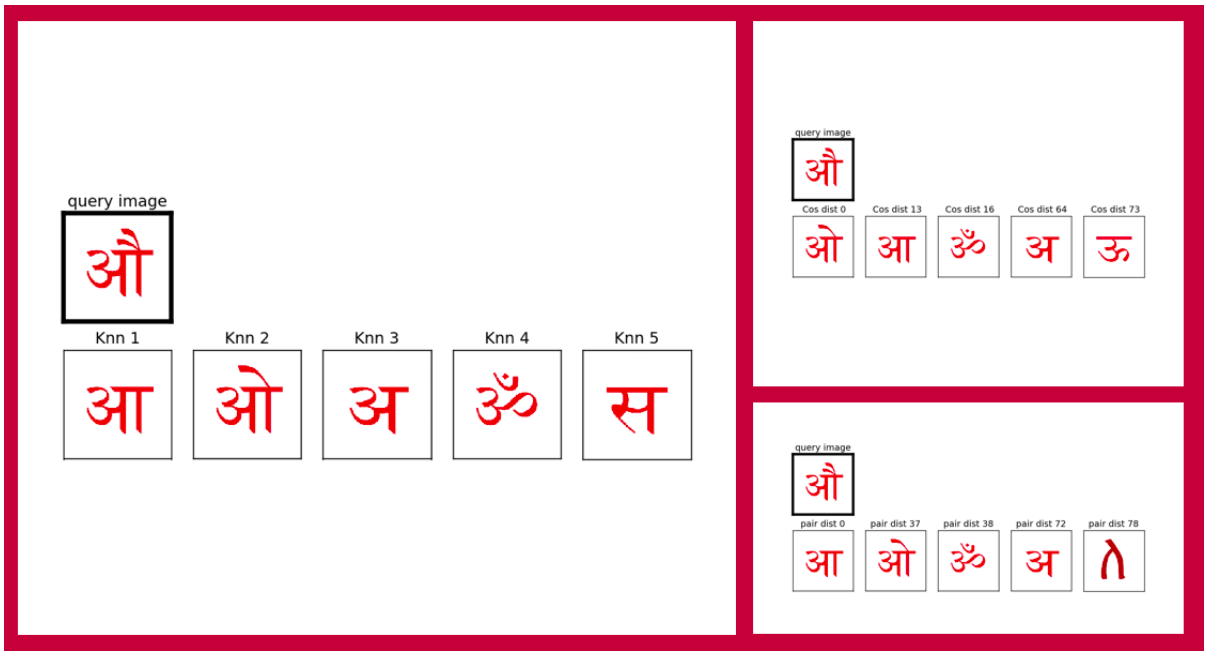


Figure 18. Similar images fetched using autoencoders. Distance metrics made a difference.

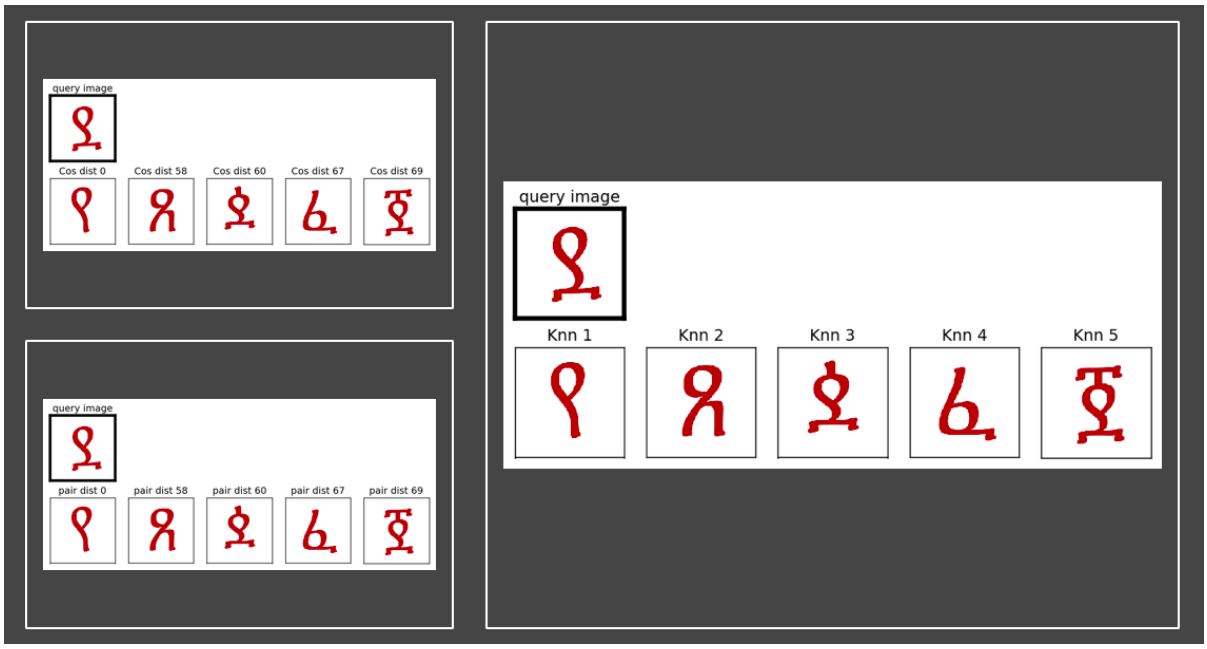


Figure 19. Similarity using Autoencoders. Distance metrics had no effect.

LOCAL BINARY PATTERN

These are image descriptors which indicate pixel wise variation of the input image. When comparing the character images, the factors that matter the most are edges, corners and lines. These give the best description of the input image in those terms.

To compute the Local Binary Pattern (LBP), central pixel is compared with the surrounding 8 pixels. If central pixel value is greater than the neighbour pixel, LBP is taken as 1 else it is 0.

Uniform LBP

An LBP is considered uniform if there are only 2 0-1 transitions. Examples are provided in Table 5.

Table 4. LBP Computation

Case	LBP
Central Pixel value < Neighboring pixel	0
Central Pixel value > Neighboring pixel	1

Table 5. LBP Types

LBP	Example
Uniform	1001000
Non-Uniform	1001011

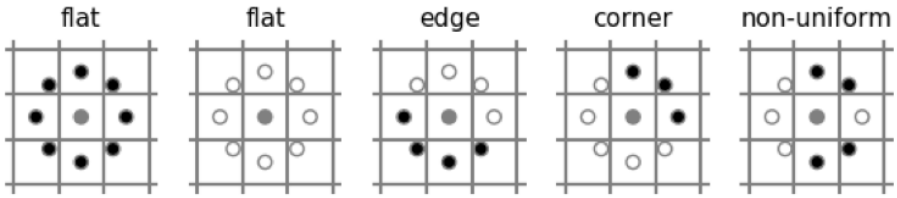


Figure 20. LBP variations.

Table 6. LBP Methods

LBP method	ror	default	uniform	non uniform	var
Grey scale invariance	Yes	Yes	Yes	Yes	No
rotation invariance	Yes	No	Yes	No	Yes
Additional features			improved invariance and finer quantization		variance measure

In this thesis, we analysed LBP using Scikit-image library. It supports 4 types of local binary pattern i.e. default, ror, uniform, var. Difference between these is as shown in table 6. All 4 types have been experimented on the dataset. These are shown in Figures 21-24.

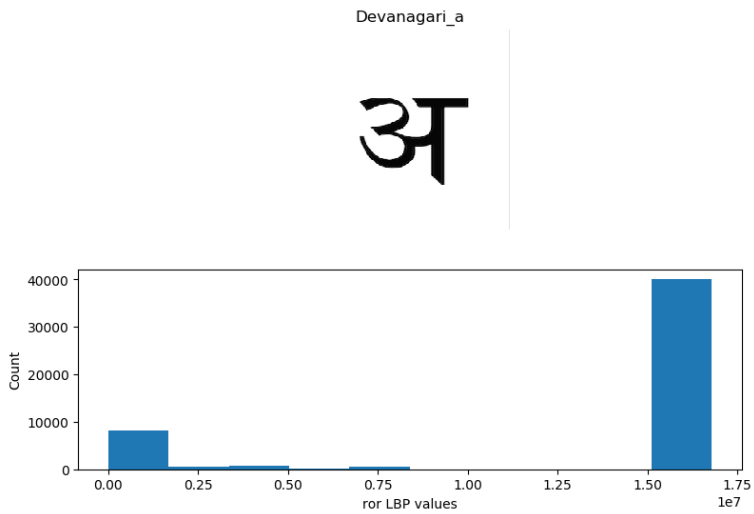


Figure 21. ror LBP values for Devanagari character.

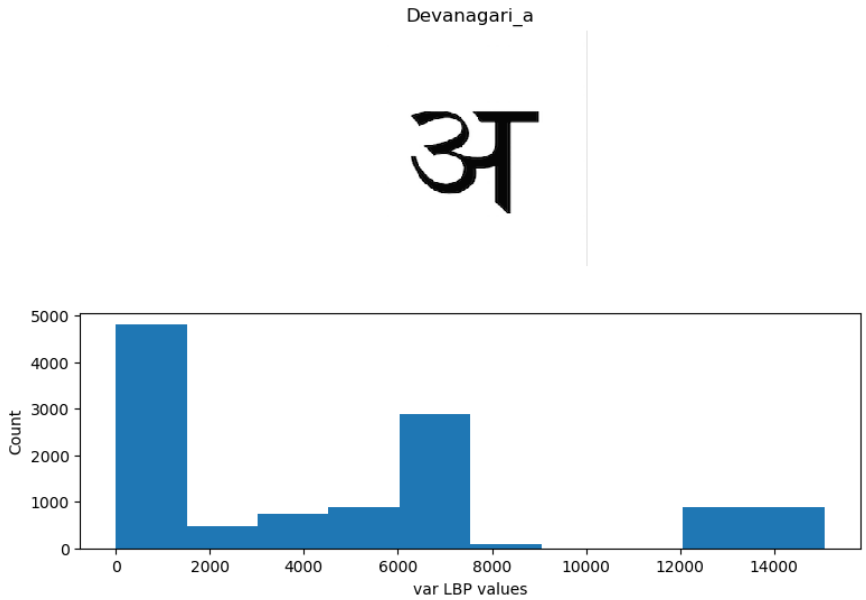


Figure 22. var LBP values for Devanagari character.

Uniform LBP works for few sets of data. It is initially used for the texture recognition problems. Another challenge that was there in the experimentation was the inconstancy in the

dataset. The character sets were not of equal dimensions, though the image dimensions were the same. This let to erroneous results.

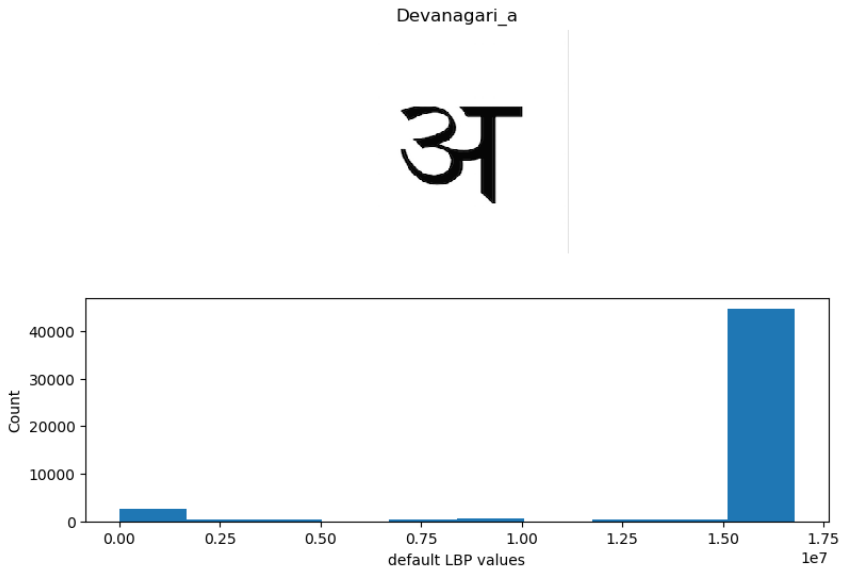


Figure 23. Default LBP values for Devanagari character.

SIMILARITY LEARNING

The features which were generated needs comparison and ranking. KNN and K means are the clustering algorithms which sort the data into groups. Similarity learning is another approach to do the same job. Different distance metrics are used to compute the scores and rank the character images.

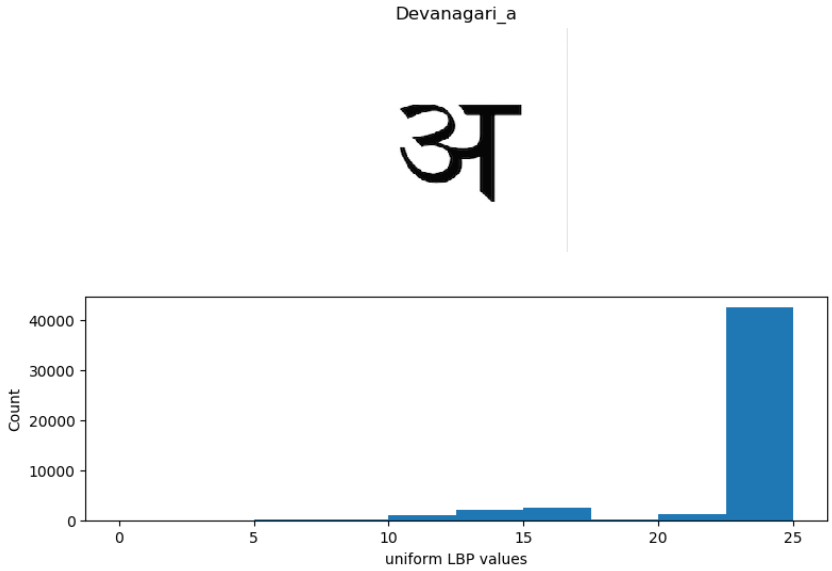


Figure 24. Uniform LBP values for Devanagari character.

Comparing the feature distance is the first approach. Here we have used: Euclidean, Manhattan, Cosine, Chebyshev, Correlation, and pairwise distance. Also, histograms have been plotted based on the core features. The distance between the histograms are compared using histogram distance measures from open cv library.

Different distance metrics tested on the dataset have been tested and are as shown in the figure 25-28. Based on the observation from the plots, it can be seen that correlation distance has less variance and a good range. The advantage of having a good range is that it gives a better estimation to understand the language similarity.

For a good distance metric, it should satisfy the following criteria:

1. $S(A,B) \geq 0$ or $D(T(A),T(B)) \geq 0$,
2. $S(A,B) = 0$ or $D(T(A),T(B)) = 0$ if, and only if, $A = B$,
3. $S(A,B) = S(B,A)$ or $D(T(A),T(B)) = D(T(B),T(A))$,
4. $S(A,C) \leq S(A,B)+S(B,C)$ or $D(T(A),T(C)) \leq D(T(A),T(B))+D(T(B),T(C))$.

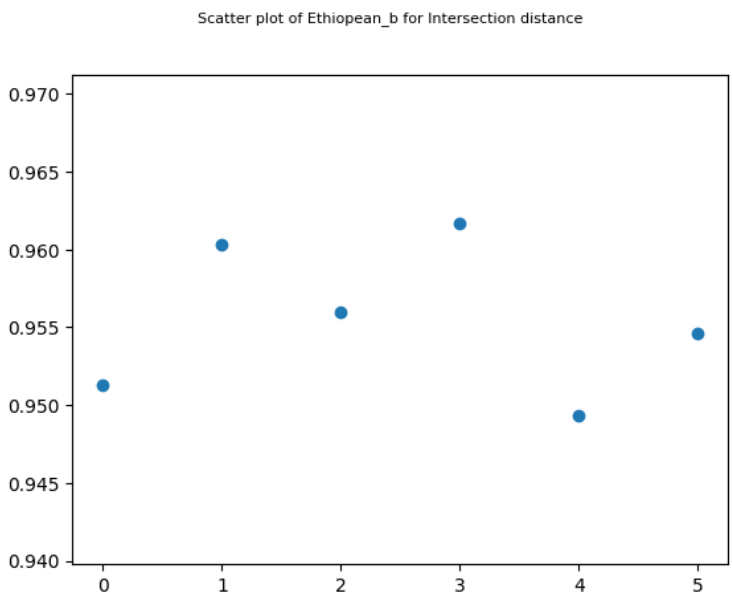


Figure 25. Scatter plot for Ethiopian script for Intersection distance.

Usually all the metrics follow the first 3 rules. The last rule, which is also known as the triangular inequality is not applicable to all the distance metrics. In this thesis, the following histogram distance metrics [1] are considered:

Histogram Intersection Distance

$$D(S,M) = \sum_i \min(S,M) \tag{1}$$

Log Likelihood Distance

$$\mathrm{L(S,M)} = -\sum_i S_i log(S_i, M_i) \tag{2}$$

Chi Square Distance

$$X^2(\mathrm{S,M}) = \sum_I \frac{(S_i - M_i)^2}{S_i + M_i} \tag{3}$$

Correlation Distance

$$C(S_i, M_i) = \frac{\sum_i (S_i(I) - S_{\bar{i}})(S_i(I) - M_{\bar{i}})}{\sqrt{\sum_I (S_i(I) - S_{\bar{i}})^2 (S_i(I) - M_{\bar{i}})^2}} \text{ Where } S_{\bar{i}} = \frac{\sum_i (S_i(I))}{N} \tag{4}$$

These distance metrics are compared by testing on the language scripts as shows below:

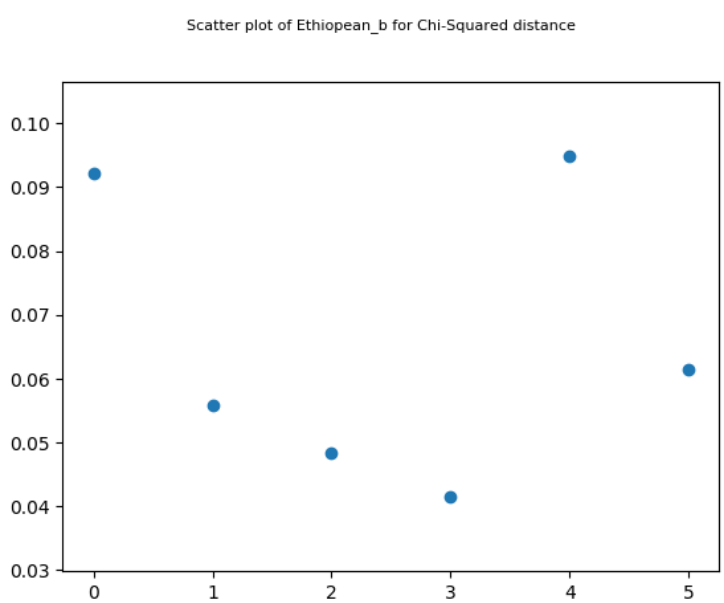


Figure 26. Scatter plot for Ethiopian script for Chi-square distance.

The distance comparison experiments helped to understand the range of the distance output, the scattering each distance produces and the data distribution. If the data is not too scattered, it will be easier to classify and make groups.

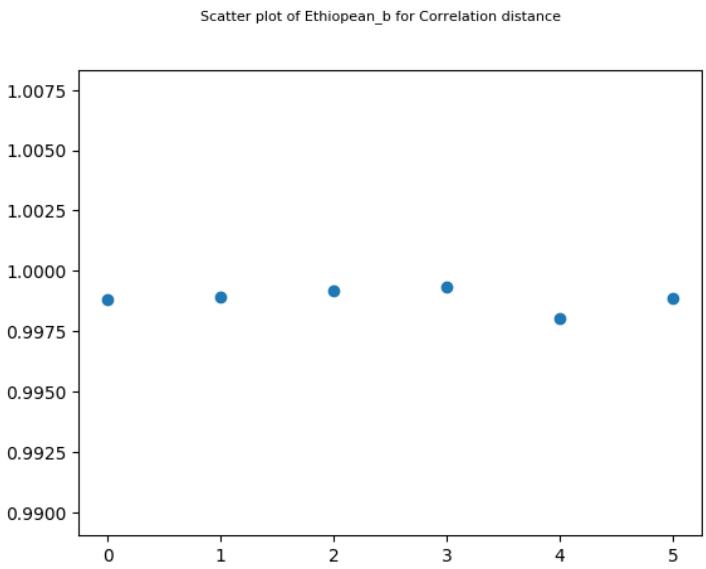


Figure 27. Scatter plot for Ethiopian script for correlation distance.

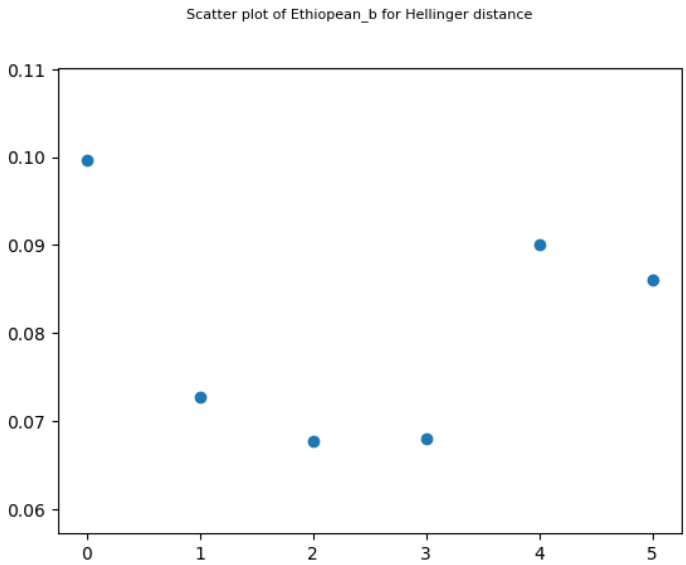


Figure 28. Scatter plot for Ethiopian script for Hellinger distance.

CHAPTER 5

RESULTS & DISCUSSION

With the basic steps of computing the PCA, an attempt was made to find the core components of a language script. The primary component and the component wise variation was a collection of characters. This gave a hint for our further analysis. When the images are studied closely, we get a hint of the way a language script unfolds itself through the characters distributed over multiple components.

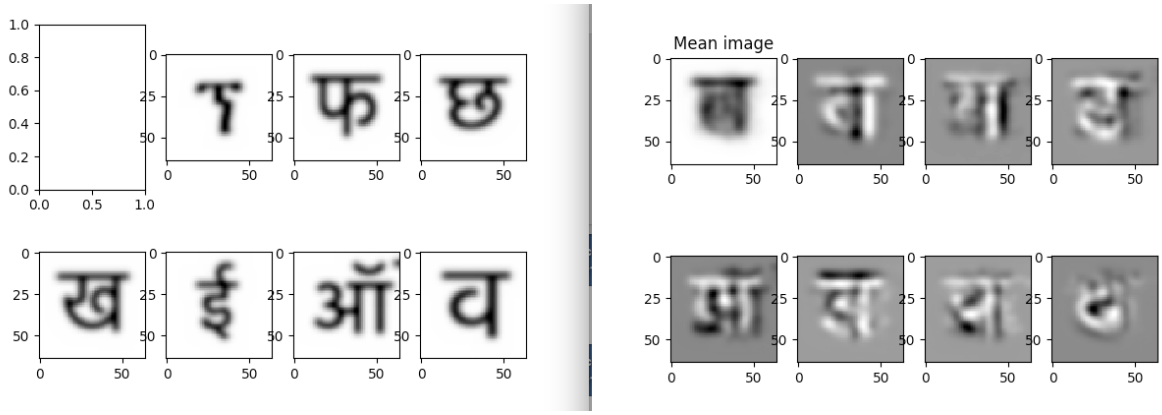


Figure 29. Image unfoldment as seen for the data set of Devanagari and Ethiopic characters.

As seen in the Figure 29, the mean image indicates that the core feature of the characters in the dataset contains a horizontal line in the top, vertical line on to the right and a blob of data attached to the vertical line. In the first primary component, which is an image on to the right of the mean image, it takes the shape of the Devanagari character, Ba. Which has a vertical line, a horizontal line and a circle attached to the vertical line.

In the next image on to the right, the vertical and horizontal line are retained. The central blob takes the form of a vertical and horizontal curve. The variation continues thus forth. From this data, an attempt was made to plot the mean and the variance histogram. A crude analogy was that if mean values of 2 language scripts are same or similar, they share a

common structure. No distance measures were used. Since the visual histogram comparison didn't help to a greater extent, next set of feature extraction algorithms were used. The results of the plotted histogram can be seen in Figure 30.

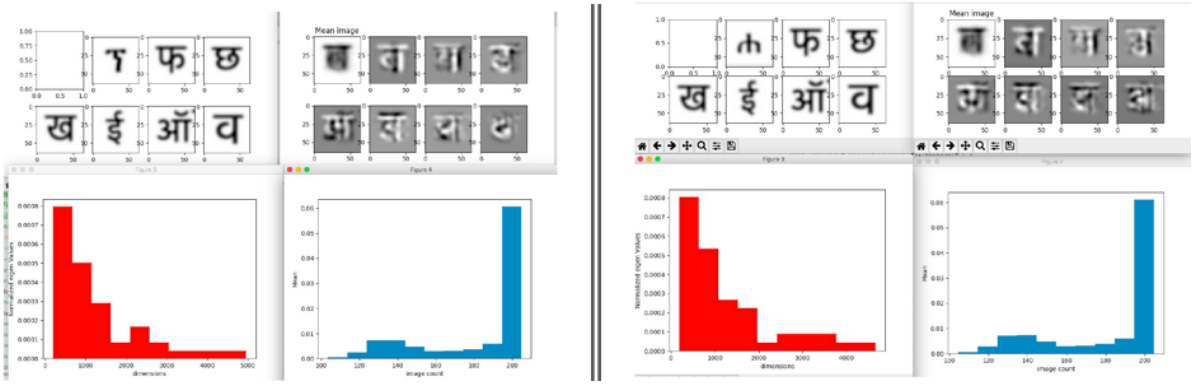
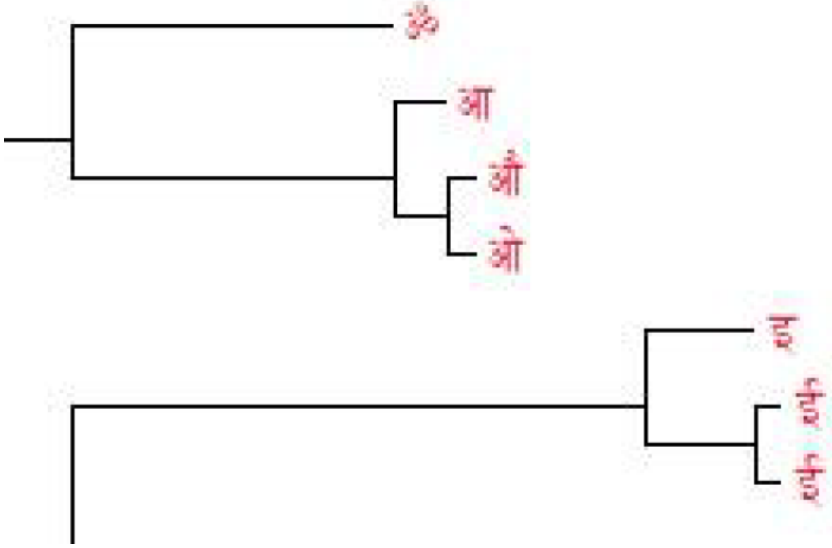


Figure 30. An attempt to compare mean and variance of different language scripts. The first character in the top left of the query image is different. Yet the principal components are the same.

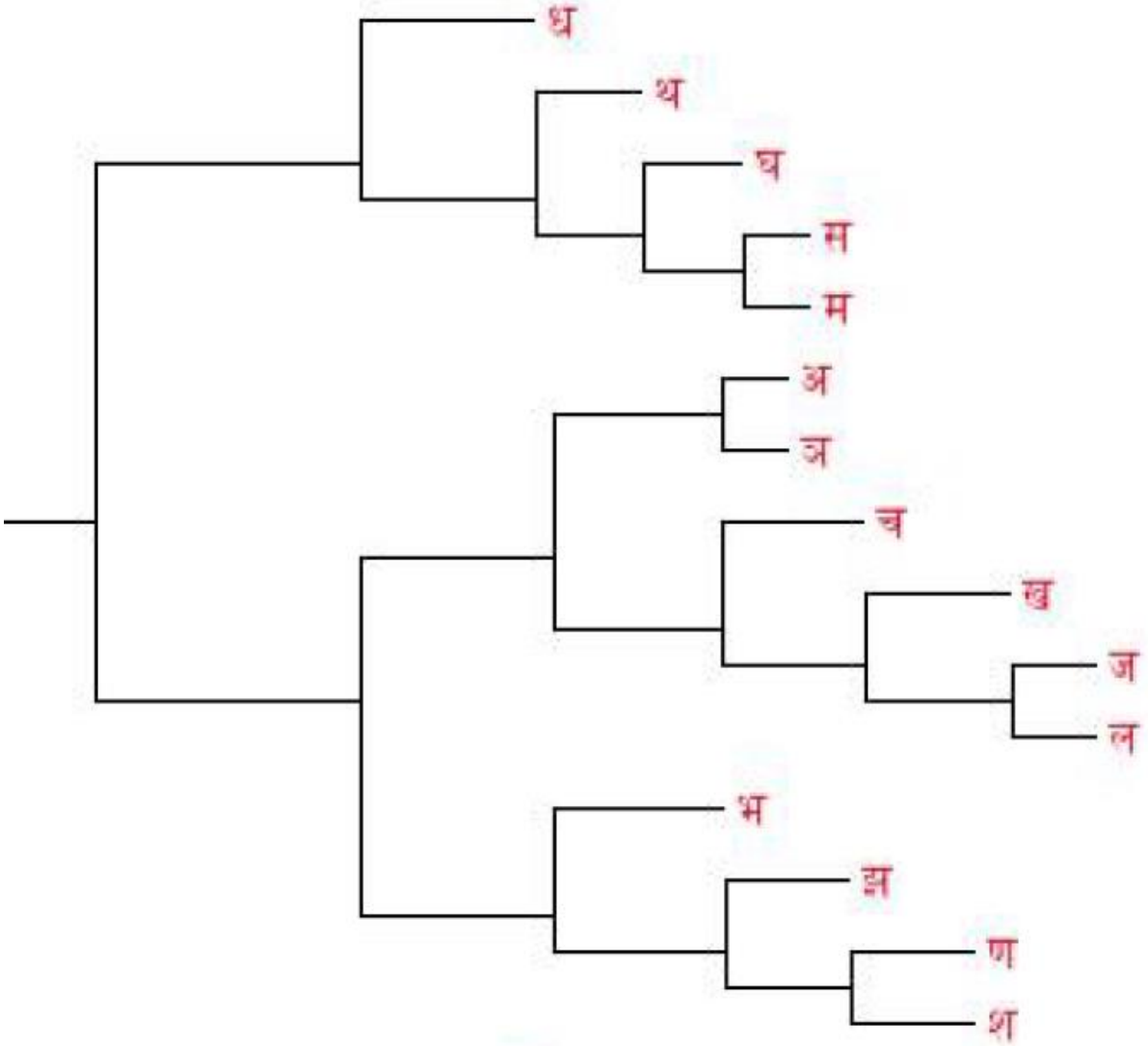
This form of structure enhancement over the higher component data obtained in Figure 29, can be utilized in the form of a clustering algorithm to generate a hierarchy tree.

Owing to the poor image quality, this output might not seem appealing. This was tested across multiple scripts. It proves good when the language character hierarchy is generated. The PCA data thus obtained was used with k means algorithm and a horizontal clustering tree was obtained. Figure 31 and 32 show the same.

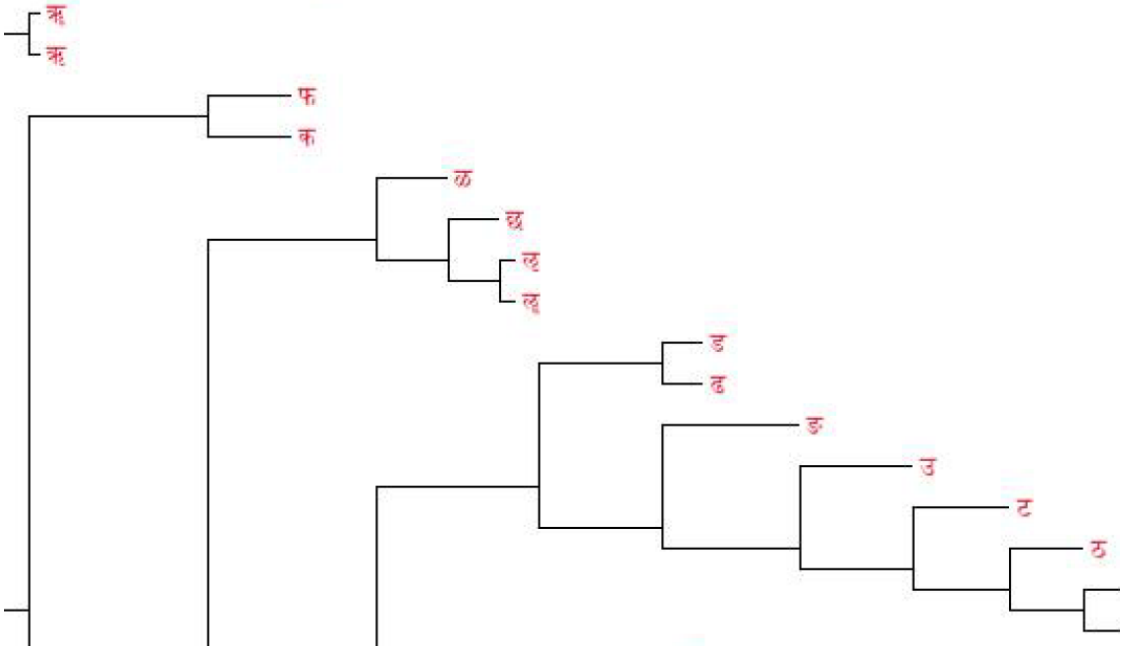
Figure 31. (a) - (d) Parts of the hierarchy tree zoomed in. (e) The entire character hierarchy tree for the Devanagari script.



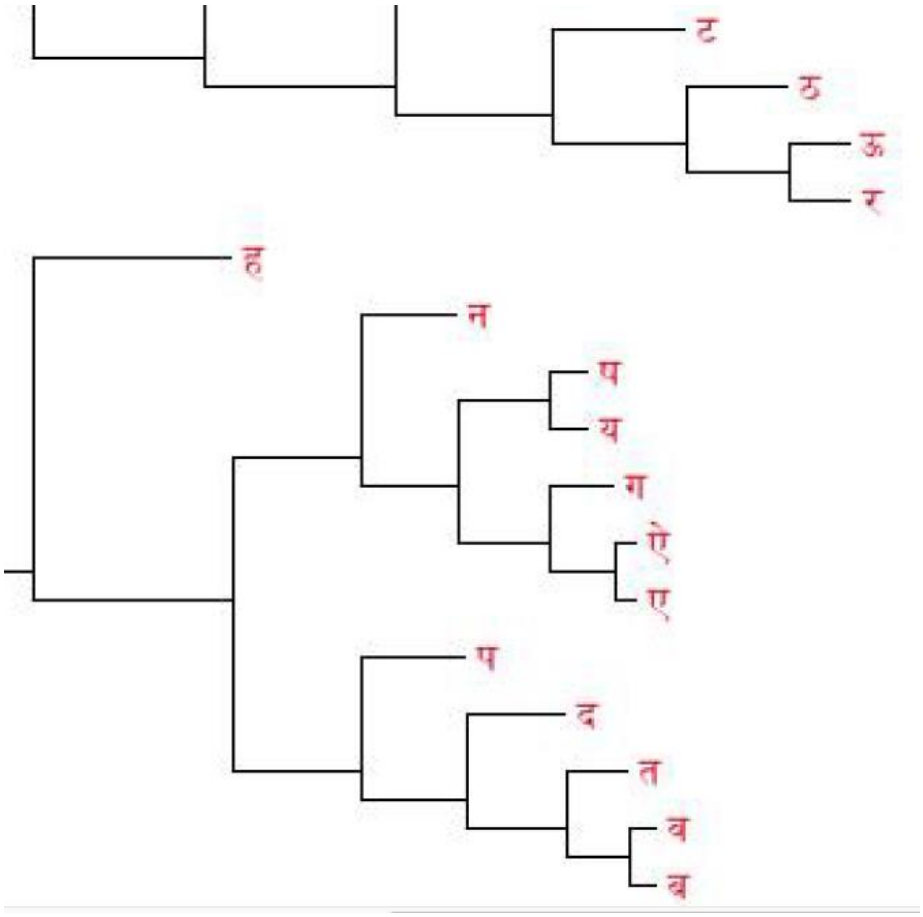
31(a)



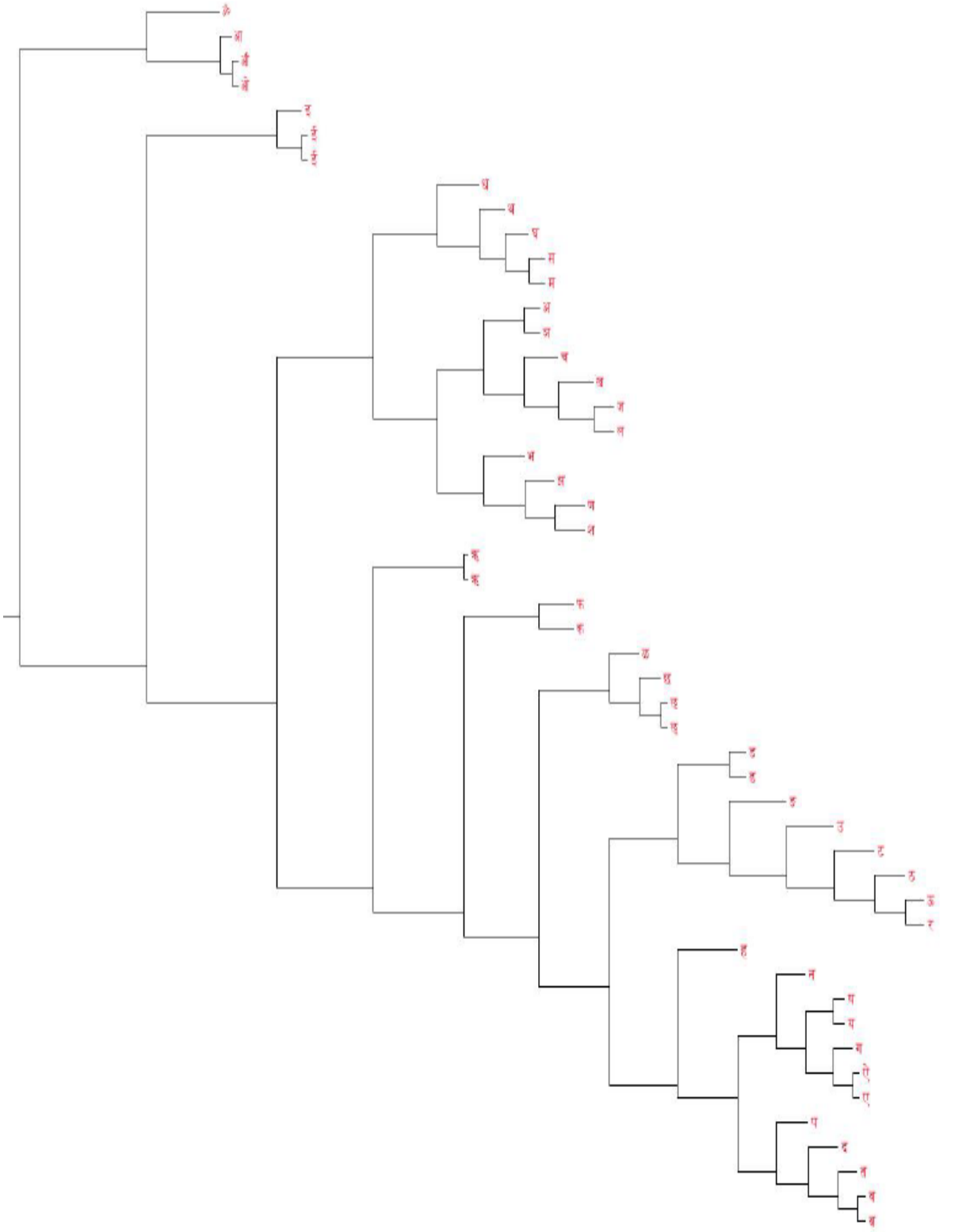
31(b)



31 (c)

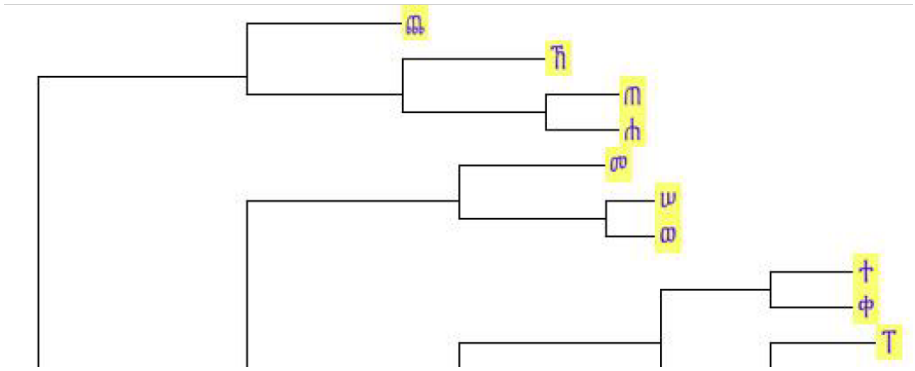


31 (d)

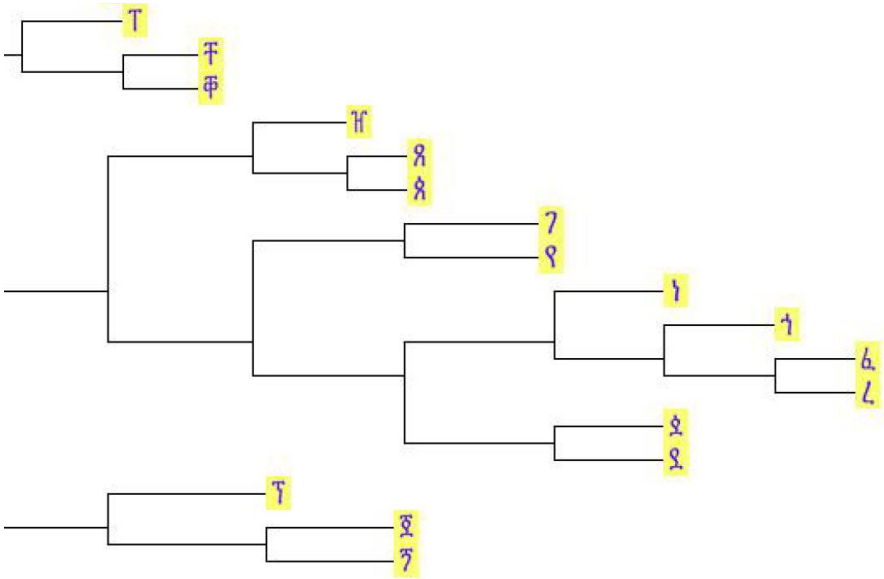


31 (e)

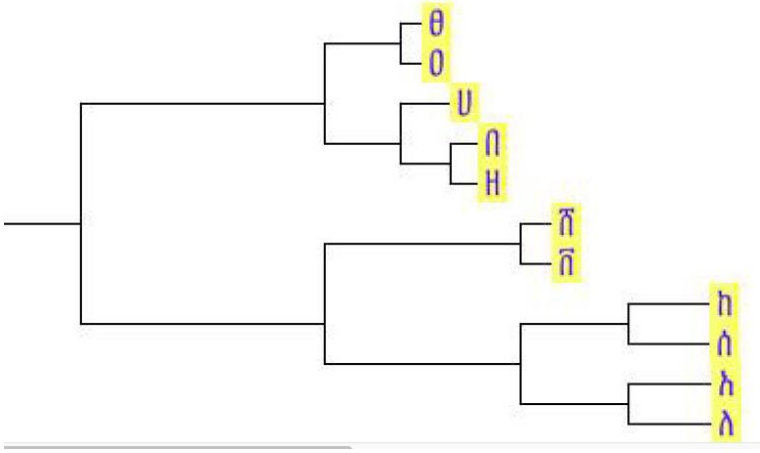
Figure 32. (a)-(c) zoomed in images of the Ethiopic tree architecture. (d) Whole tree structure for the Ethiopic characters.



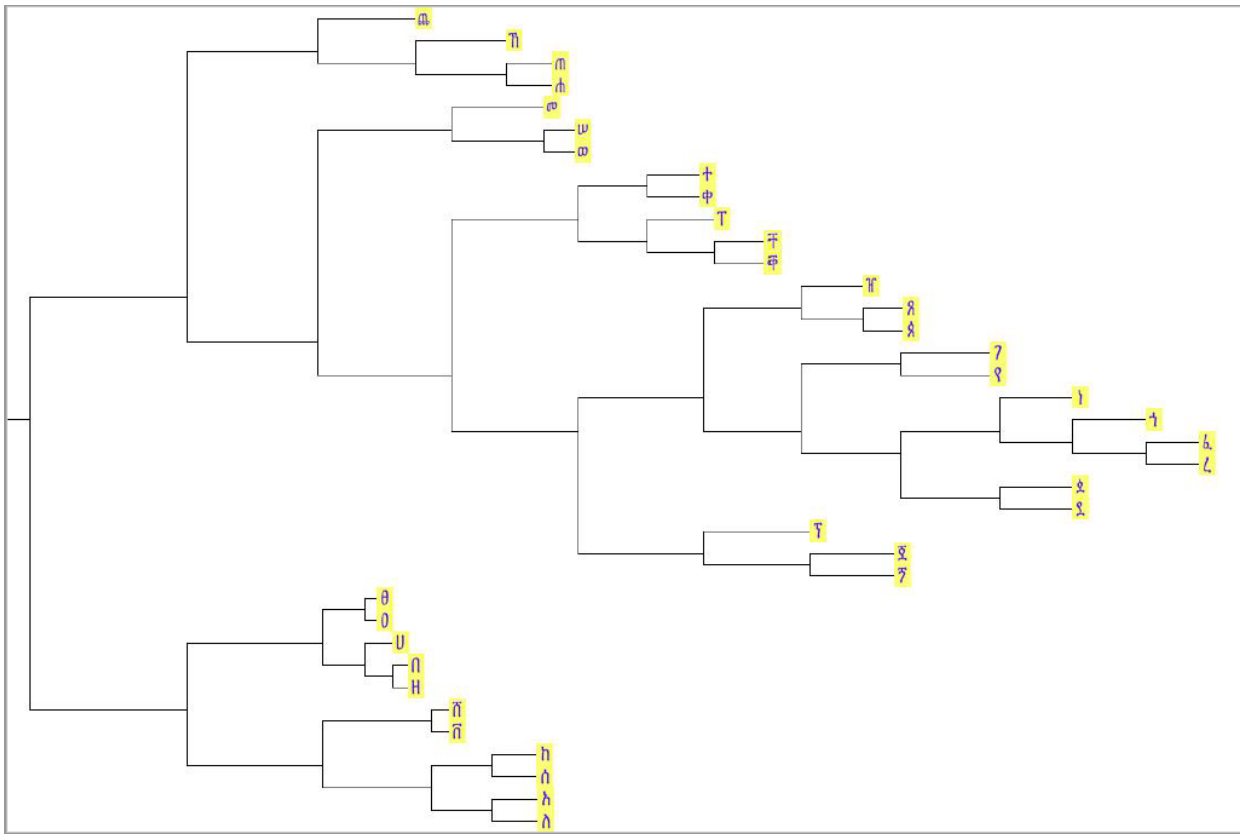
32 (a)



32 (b)



32(c)


ՄՀ
ՄՄ
ՄՍ
ՄՎ
ՄՁ
ՄՂ
ՄՃ
ՄՄ
ՄՅ
ՄՇ
ՄՈ
ՄՉ
ՄՊ
ՄՋ
ՄՌ
ՄՍ
ՄՎ
ՄՏ
ՄՐ
ՄՑ
ՄՓ
ՄՔ
ՄՕ
ՄՖ
ՄԱ
ՄԲ
ՄԳ
ՄԴ
ՄԵ
ՄԶ
ՄԷ
ՄԸ
ՄԹ
ՄԺ
ՄԻ
ՄԼ
ՄԽ
ՄԾ
ՄԿ
ՄՀ
ՄՂ
ՄՄ
ՄՆ
ՄՇ
ՄՈ
ՄՊ
ՄՋ
ՄՌ
ՄՍ
ՄՎ
ՄՏ
ՄՐ
ՄՑ
ՄՓ
ՄՔ
ՄՕ
ՄՖ

32 (d)

The results from Autoencoder, were analyzed using a t-SNE plot. Different combination of the dataset was used as part of the experiment. The spatial distribution obtained, was a good indication of the distribution. These are shown in Figure 33.

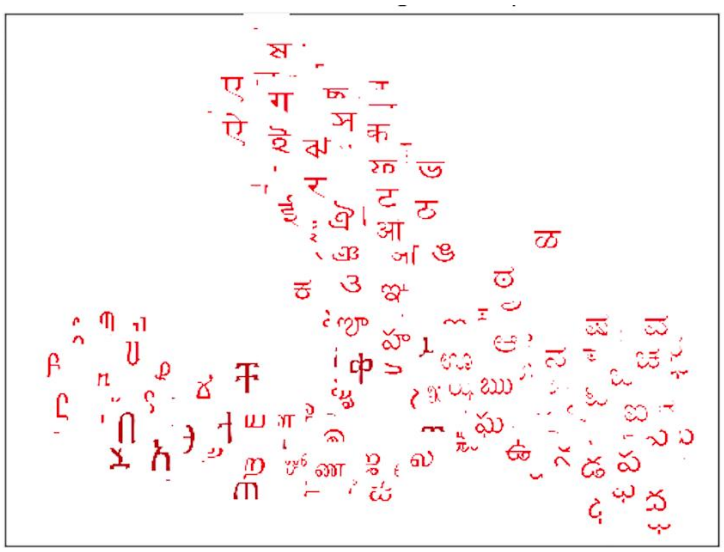


Figure 33. t-Sne plot of all the images in the dataset.

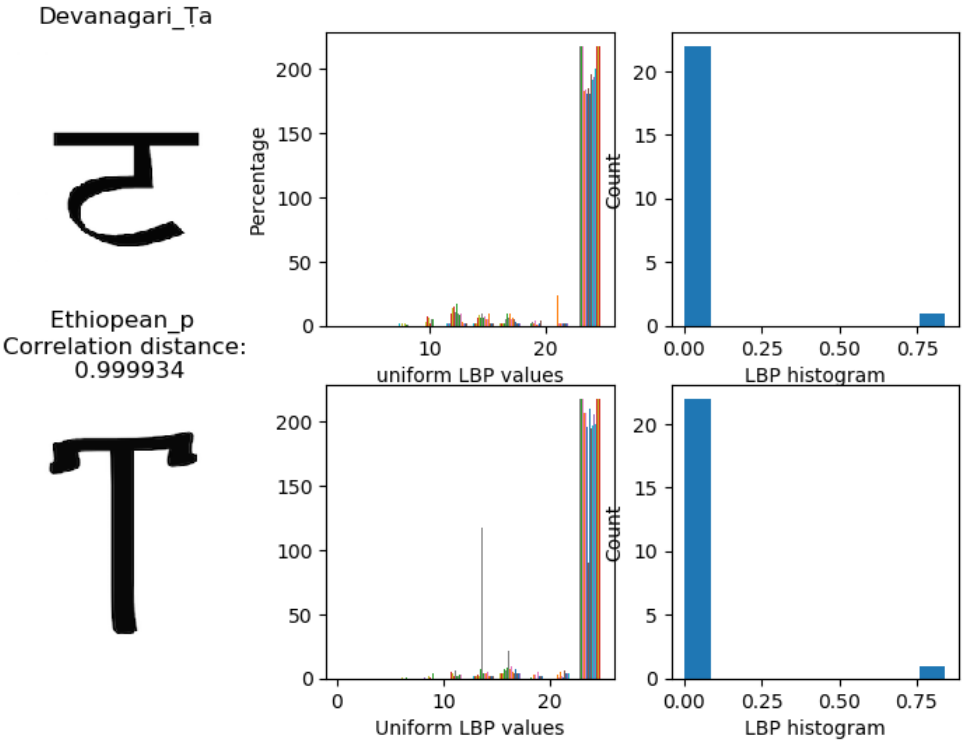


Figure 34. LBP computation and similarity measurement for Devanagari Ethiopian characters.

Results from the uniform Local Binary Pattern was used along with the correlation distance. Each character in the query language script was used to compare with all the characters in the database. The outcome of the similairy learning, was filtered based on at least 50% similarity. Count of the character similarity betweeen each language is monitored. Percentage match is defined as total character matches in language B from language A to the total characters in the language B. The results were obtained as shown in Figure 34 and 35.

FURTHER ENHANCEMENTS

The experiments were performed on a limited set of language scripts, with particular focus on the alphabets. The same can be extended to other modern-day writing systems. In any given Machine Learning approach, the biggest problem is the data set. Experimentation with a better dataset would improve the performance.

Algorithms used for the thesis can be improved in terms of the efficiency. More accurate the feature extraction algorithms, better would be the results.

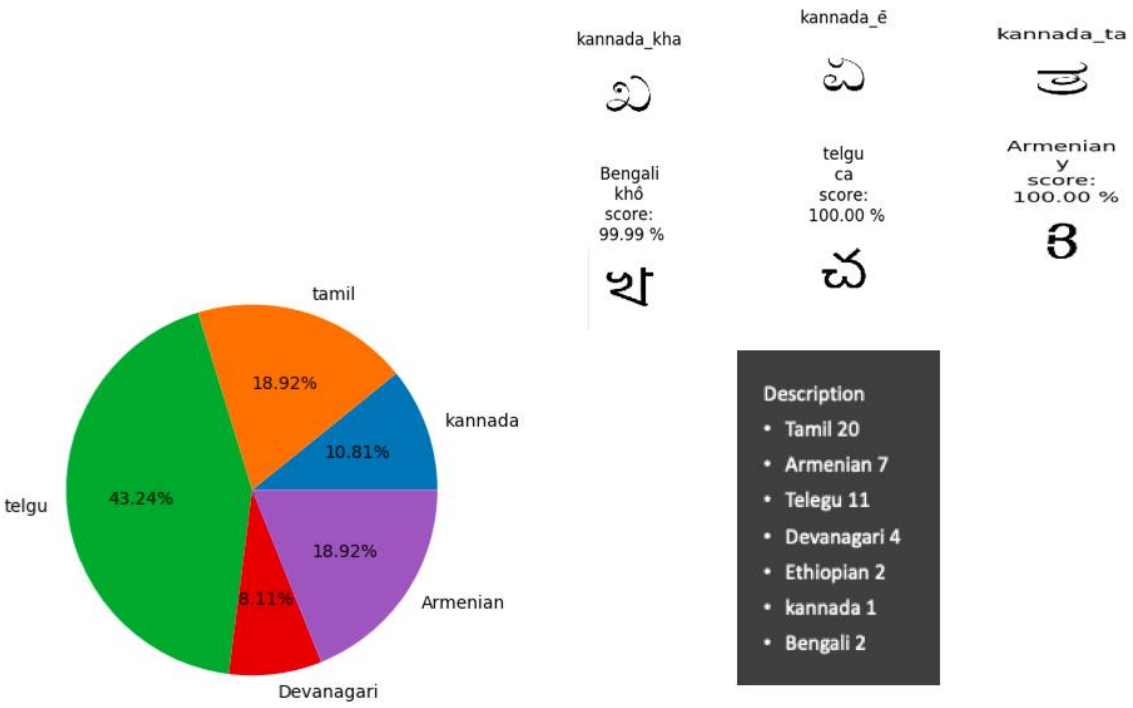


Figure 35. Character to character match measurement using the cut off of 50% for the Kannada language script.

SUMMARY

Experiments helped to get an understanding of the machine learning technique. Local Binary Pattern was a technique that described the data to its best in terms of features. Autoencoders gave the best performance in terms of clustering the data into different groups. K means was the best clustering algorithm. Chi-squared distance performed better than all the other distances. Through the experiments carried out for this thesis we were able to:

- generate a character tree based on the structural similarity using PCA and KNN
- obtain the percentage match between characters
- find the communality/similarity percentage between scripts based on the structural aspects.

The results are based on the Machine Learning approach. This is a pure academic analysis and do not represent any cultural, religious, regional, political or any other views.

REFERENCES

[1] J. DEFRANCIS, *Visible Speech: The Diverse Oneness of Writing Systems*, University of Hawai'i Press, Honolulu, Hawai'i, 1989.

[2] B. PRIMUS, *A Featural Analysis of the Modern Roman Alphabet*, Benjamins, Amsterdam, The Netherlands, 2004.

[3] WIKIPEDIA, *Brahmic scripts*. Wikipedia, https://en.wikipedia.org/wiki/Brahmic_scripts, n.d., July 2019.

[4] R. RAO, *Probabilistic analysis of an ancient undeciphered script*, IEEE Comput. Soc. J., 43 (2010), pp. 76–80.

[5] S. M. R. ALDURI, *Evolution of Telugu character graphs*. Telugu Language & Literature, https://www.engr.mun.ca/~adluri/telugu/language/script/script1d.html, n.d., March 1998.

[6] A. PRADESH. *Telugu is 2,400 years old, says ASI*. News Release, Apr. 29, 2011. https://www.thehindu.com/todays-paper/tp-national/tp-andhrapradesh/Telugu-is-2400-years-old-says-ASI/article14898202.ece, n.d.

[7] Y.-H. LIU, C.-C. LIN, AND F. CHANG, *Language identification of character images using machine learning techniques*, in Eighth International Conference on Document Analysis and Recognition (ICDAR'05), IEEE, Piscataway, New Jersey, pp. 1–5.

[8] M. S. PATEL AND S. L. REDDY, *An impact of grid based approach in offline handwritten Kannada word recognition*, in 2014 International Conference on Contemporary Computing and Informatics (IC3I), IEEE, Piscataway, New Jersey, pp. 630–633.

[9] J. J. LIM, R. SALAKHUTDINOV, AND A. TORRALBA, *Transfer learning by borrowing examples for multiclass object detection*, in NIPS'11 Proceedings of the 24th International Conference on Neural Information Processing Systems, Neural Information Processing Systems Foundation, San Diego, CA, 2011, pp. 118–126.

[10] N. HALKO, P.-G. MARTINSSON, AND J. A. TROPP, *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions*. arXiv, https://arxiv.org/abs/0909.4061, n.d., December 2010.

[11] M. TURK AND A. PENTLAND, *Eigenfaces for recognition*, J. Cogn. Neurosci., 3 (1991), pp. 71–86.

[12] A. GORDO, J. ALMAZAN, J. REVAUD, AND D. LARLUS, *Deep image retrieval: Learning global representations for image search*. arXix, https://arxiv.org/abs/1604.01325, n.d., July 2016.

[13] K. SIMONYAN AND A. ZISSERMAN, *Very deep convolutional networks for large-scale image recognition*. arXiv, https://arxiv.org/abs/1409.1556v6, n.d., April 2015.

[14] S. B. PATEL AND A. SAMPAT, *Semantic image search using queries*, Stanford University, Stanford, California, 2015.

[15] T. OJALA, M. PIETKANINEN, AND T. MAENPAA, *Multiresolution gray scale and rotation invariant texture classification with local binary patterns*, IEEE Trans. Pattern Anal. Mach. Intell., 24 (2002), pp. 971–987.

[16] L. ZHANG, R. CHU, S. XIANG, S. LIAO, AND S. T. LI, *Face detection based on multi-block LBP representation*, in International Conference on Biometrics (ICB 2007), Springer, Berlin, pp. 11–18.

[17] B. K. WELLS AND A. FULS, *The Indus script*. Archäoastronomie, http://www.user.tu-berlin.de/fuls/Homepage/indus/menueindus.htm, n.d., n.d.

[18] BRENDENLAKE, *Omniglot database*, GitHub, https://github.com/brendenlake/omniglot, n.d., January 2019.

[19] X. NÈGRE, *Multilingual keyboard*. Lexilogos, https://www.lexilogos.com/keyboard/, n.d., n.d.

[20] SCI KIT LEARN, *Mini batch k-means*. Sci Kit Learn, https://scikit-learn.org/stable/modules/clustering.html#mini-batch-kmeans, n.d., n.d.